WALKTHROUGH PREPARATION · STUDY GUIDE

The Governed Data Product for AI, explained step by step

Every step from raw files to an LLM that is treated as a dumb and dangerous client: what was built, why, what it costs, what the evidence is, and where it is weak.

Berka (PKDD'99) take-home | PostgreSQL | Data quality | Semantic layer | Row level security | LLM as untrusted client

Describes the **delivered version**: branch **delivery-freeze-2026-10-28**, commit **fc0c281** (code b5b18f2), 2026-10-28.
Work done **after** delivery (D028, D029, branch ccr-46f6f45a-t9wtd4) is marked as such wherever it appears.
Every figure comes from the delivered evidence files or a query run on 2026-11-04; nothing is from memory.

Contents

0 Start here 3
- What is in the study pack 3
- A four-day plan to Monday 3
- Which version you are defending 3
- The story in 60 seconds 4
- Figures you should know without looking 4
- How evidence is labelled in this pack 4

1 The brief, and the way of working 5
- What the brief asked for 5
- The way of working, as nine rules 5
- How AI tools were used 5
- Why this stack 6

2 The architecture in one picture 7
- What runs, in order 7
- One question, end to end 7
- Three contracts that hold the layers together 8

3 Step 1 — Acquire and ingest (Bronze) 9
- What it does 9
- Why it is built this way 9

4 Step 2 — Profile before writing rules 10
- What it does 10

5 Step 3 — Clean and transform (Silver) 11
- The cleaning ledger, steps C03–C12 11
- The decisions that matter most 11

6 Step 4 — Data quality rules and reconciliation 12
- The rule set 12
- What each severity does 12
- The 14 defect types 12
- The firing warn rules, and what they mean for the answers 13
- Proving the rules fire: the defect injector 13
- Reconciliation: proving nothing was lost 13

7 Step 5 — The model (Gold) 14
- The objects and their grains 14
- Why a star with a bridge 14
- One definition, decided here 14
- The month-end balance, and its known limitation 15
- Verified after every build 15

8 Step 6 — Reference answers, the marking scheme 16

9 Step 7 — The semantic layer 17
- What is in the file 17
- The query rules the model is given 17
- Validated against the live database 17
- What makes the layer AI-ready, specifically 18

10 Step 8 — Entitlements: where the filter sits 19
- The three answers the brief asked for 19
- Four sources of access, one shape 20
- Measured, by logging in as each role 20
- Tested by attacking it 20

11 Step 9 — Authentication: two factors, in the database 22
- How it works 22

12 Step 10 — The LLM as a dumb and dangerous client 23
- The threat model 23
- The defence, layer by layer 23
- The observed failure that proves the point 23
- The local model shows what "dumb" looks like 24

13 Step 11 — Asking the questions and evaluating 25
- The runner 25
- The evaluation 25
- The results 25

14 Step 12 — Demo, deliverables and the evidence chain 26
- How it is shown 26
- The deliverables and where their numbers come from 26

15 The ten questions, one by one 27
- Q1 — How many clients do we have per region? 27
- Q2 — What is the total amount of loans granted per year? 27
- Q3 — What percentage of our loans are problematic? 27
- Q4 — What is the average loan amount per card type of the account owner? 28
- Q5 — Which 10 districts had the highest total incoming transaction amount in 1997? 28
- Q6 — How many new accounts were opened per month in 1996? 29
- Q7 — Average age of the account owner at loan start, per loan status 29
- Q8 — Gender split of account owners holding a gold card 30
- Q9 — Average account balance at the end of December 1998 30
- Q10 — Which statement frequency has the highest share of problematic loans? 31

16 What breaks at 100,000× the volume 32

17 Question bank 33
- Brief, scope and design 33
- Data quality and cleaning 33
- Model and SQL 33
- The ten questions 34
- Semantic layer, LLM and evaluation 34
- Entitlements and authentication 35
- Testing and operations 35
- When you do not know 35

18 Decision register 36
- Decisions D001–D012: platform, data, model, security foundations 36
- Cleaning decisions (D013-D020) 38
- Privacy at client grain (D021-D022) 40
- Identity, access and authentication (D023-D027) 40
- After delivery: D028–D029 (not in the delivered version) 41
- Semantic correctness and balance integrity (D028-D029) 41

19 Assumption register 43
- Question interpretation 43
- Pipeline and environment 43
- Cleaning and extraction 44
- Privacy and identity 44
- Identity, access and authentication 45
- Added after delivery (not in the delivered version) 45

20 Glossary and where to find things 46
- Terms 46
- Codes worth knowing 46
- Where to find things in the repository 46

0 Start here

This guide is the long form of the study pack. It explains every step of the project, from the eight raw files to an AI assistant that is treated as a dumb and dangerous client, and for each step it gives the decision, the reason, the cost, the evidence and the weak spot you should expect to be asked about.

What is in the study pack

File	Use it for	When
01 Study Guide (this file)	Understanding every step deeply; the decision and assumption registers; the question bank	Days 1 to 3
02 Lessons Learned and Self-Audit	The mistakes and sloppy spots, each verified, with how to talk about them	Day 3, then skim again on day 4
03 Walkthrough Deck (.pptx and .pdf)	The 30 minutes you present in the 45-minute walkthrough, with speaker notes	Rehearse days 2 to 4
04 Cheat Sheet	One page of figures and one-liners, to print	Day 4 and Monday morning

The brief's follow-up is a **45-minute technical walkthrough in which you defend your design choices, with 15 minutes of questions**. The deck is sized for about 30 minutes of talking. Everything else in the pack exists so that the 15 minutes of questions go well.

A four-day plan to Monday

Day	Study	Practise
Thursday	Chapters 0 to 3 (story, way of working, architecture) and steps 1 to 5 (ingest, profile, clean, rules, model). The lessons-learned summary table	The 60-second story out loud, three times. Draw the six layers from memory
Friday	Steps 6 to 12 (reference answers, semantic layer, security, authentication, the LLM path, evaluation, demo). The ten questions	Present the deck once, untimed. Explain "where the filter sits" without slides
Saturday	The full Lessons Learned document. The question bank (chapter 17)	Deck timed at 30 minutes. Answer 15 random questions from the bank aloud. Practise the five "raise it yourself" items
Sunday	Cheat sheet; skim the decision and assumption registers	Dry run of the demo on the laptop (checklist below). One full mock walkthrough with somebody asking questions. Then stop
Monday	Cheat sheet only	Arrive with the demo already running

Demo checklist for Sunday evening

- Start the PostgreSQL service and run `scripts\windows.ps1 test` once: you want to know it is green before you are in the room.
- The demo delegation (`rm_south` covering client 3763) was seeded for 14 days at the last `entitlements` run and lapses around **10–11 November**. Check it with `SELECT valid_to FROM sec.access_delegation;` and re-run `scripts\windows.ps1 entitlements` the day before if needed. After it lapses, `rm_south` shows 1,386 clients instead of 1,387.
- If you demo the web page with Ollama, warm the model up first (the first call loads about 1.9 GB). Expect a small model to get some questions wrong; that is part of the story, not a failure of the governance.
- Have `docs/DELIVERY_AUDIT.md` open in a tab: it is the single document that maps every claim to its evidence.

Which version you are defending

There are two lines of work, and you must not mix them up in the room.

	Delivered version	Work after delivery
Where	Branch `delivery-freeze-2026-10-28` , commit `fc0c281` (code `b5b18f2`), built on the Windows laptop, PostgreSQL 18.6	Branch `ccr-46f6f45a-t9wtd4` , commits `49059c0` , `a308fc1` , `6943873` (2026-10-28, not merged into the delivery)
Month-end balance	Same-day ties broken by the **highest** `trans_id` (A14, D019). Known limitation, disclosed	Closing posting = end of the running-balance chain, ambiguous days marked (D028)
Q9	**43,808.94** CZK	43,810.47 CZK
Material balance breaks	55 accounts	14 accounts (41 of the 55 were artefacts of the tie-break)
Meaning of generated SQL	Guard checks **shape only**; intent checker is a **prototype** (`scripts/ir_check_prototype.py`)	Intent (IR) verified before and after execution in both CLI and web (D029)
LLM evidence	Claude Sonnet 5.5, run 2026-10-25 on the older 29,500-character pack: 10/10. Ollama qwen2.5-coder:3b: 6 match, 2 partial, 2 mismatch	Claude re-run on the new pack: 10/10 verified, plus 4 extra questions 4/4
Decisions / assumptions	D001–D027, A1–A46	Adds D028, D029, A47–A49

How to handle the two versions

"What I submitted keeps the month-end calculation and its reference answers unchanged, and says plainly that the tie-break is a known limitation and that the intent checker is a prototype. After submitting I fixed both on a separate branch, and I can show you the before and after." Present the delivered numbers as the answer; present the later work as what you learned and did next. Never quote 43,810.47 as the delivered Q9.

The story in 60 seconds

Business users want to ask a bank's data questions in plain English. An AI assistant can only answer correctly if the data is clean, the words mean one thing, and every user sees only their own clients. I treated the assistant as a **dumb and dangerous client**: dumb, because it will guess at anything the data does not spell out; dangerous, because it writes code that runs against customer data. So every layer either removes a reason to guess or removes a way to do harm.

The eight raw files land verbatim, hashed. Staging types and decodes them but never repairs or drops anything; 124 rules then grade every defect as block, warn or info, and each severity has a mechanical consequence. A star schema with one definition per business term sits on top, and a validated semantic layer tells the model what every entity, metric and word means, and what it does *not* mean. The model sees only that, never the data. Its SQL is parsed and bounded, and it runs on the user's own two-factor database session, where row level security on the base tables decides which rows exist. The filter is not in the query, so nothing the model writes can remove it. Claude answered all ten questions and all ten matched hand-written reference SQL; I can show you exactly where that evidence is weaker than it sounds.

Figures you should know without looking

1,079,680
source rows in 8 files, counts match the published reference exactly

124
rules executed: 89 hand written + 35 completeness sweep; 19 fire

14
defect types: 1 block, 4 warn, 9 info

1 / 440
row quarantined / warn flags (on 352 distinct rows)

21
cleaning steps; exactly one removes a row (C14)

20
seeded defects, each caught by the rule meant to catch it

7
row level security policies on 7 core tables

10 / 10
Claude answers matching the reference (recorded 2026-10-25)

Area	Figure
Source tables	trans 1,056,320 · client 5,369 · disp 5,369 · account 4,500 · card 892 · loan 682 · order 6,471 · district 77
Reconciliation	raw = stg on 11 checks; core = stg − quarantine on 9 (7 row counts, 2 money sums: transactions 6,257,793,560.30, loans 103,261,740.00)
Model	4 dimensions from the source tables + a generated calendar (`dim_date`), 1 bridge, 2 base facts, 1 derived month-end fact (185,615 rows, 558 carried forward), 3 curated views
Semantic layer	11 entities, 105 columns, 19 joins, 12 metrics, 7 caveats, 12 query rules, 10 glossary terms; delivered pack 33,662 characters
Problematic loans	status B or D: 76 of 682 = 11.14% (B 31, D 45) — A4, awaiting business confirmation
Visible rows	`cro_global` 5,369 clients · `rm_north` 1,481 · `rm_south` 1,387 (1,386 + 1 delegated) · `rm_unassigned` 0 · `client_3764` 1
LLM runs	Claude Sonnet 5.5: 10 answered first attempt, 10 match. Ollama qwen2.5-coder:3b: 6 match, 2 partial (Q3, Q7), 2 mismatch (Q6, Q10)
Tests (delivered)	327 passed, 1 skipped (full suite at `1b837fd`); `test_auth.py` 40 passed, 1 strict xfail (`b5b18f2`); 330 collected
Deliverables	Workbook 11 tabs · DDR 2 pages · ZIP 183 members · defect register 14 rows

Numbers that differ between documents

Some delivered documents still quote older figures (137, 202 or 241 tests; "88 rules"; "PostgreSQL 16"; a 4m28s build time measured once on Linux). The authoritative figures are the ones in `docs/DELIVERY_AUDIT.md` and on this page. If someone quotes an old number back at you, say which one is current and where it is measured. Chapter 6 of the Lessons Learned document (L-14) lists every stale figure.

How evidence is labelled in this pack

The project labels every claim by how it is known, and so does this guide.

Label	Meaning
RUN	Executed: a query, a build step or a test produced it
CODE	Read in the source at the cited place
LLM	Output of the named provider and model
REC	Recommended, not implemented
ASSUMED	A business definition awaiting confirmation
POST	Done after delivery; not in the delivered version

1 The brief, and the way of working

What was asked, what was handed in for each part of it, and the working rules that explain most of the design choices before anyone asks.

What the brief asked for

The brief ("The Governed Data Product for AI") has three tasks on the public Berka (PKDD'99) Czech bank dataset and ten business questions in `questions.csv`.

Task	The brief's words	What was delivered	Where
1. Data quality and reconciliation	Rules for at least completeness, uniqueness, referential integrity, validity and cross-table consistency; defect types with the rule broken and a severity (block / warn / info); *justify the severity*	124 rules in six dimensions (the five plus reconciliation), every one with a severity rationale written in terms of the questions; 14 defect types	Workbook tabs *Defect types* and *DQ rules*; `dq/dq_rules.yaml`; `dq/results/dq_report.md`
2. AI-ready data product with a semantic layer	A data model on the cleaned data; a semantic solution to guide an AI; use an LLM with it to answer the 10 questions; submit generated SQL and result per question	Star schema in `core`; curated `ai` views; validated YAML semantic layer rendered to a context pack; Claude's SQL and results, evaluated against reference SQL	Workbook tabs *Model*, *Questions*, *Q detail*; `deliverables/model_explanation.md`; ZIP
3. Entitlement-aware access	*Explain* how each user sees only their own clients; where the filter sits; why an LLM query cannot bypass it; onboarding without code changes	Implemented, not only explained: row level security on the base tables, identity from `session_user`, two-factor login, grants derived from data, bypass tests	Workbook tab *Entitlements*; `docs/ACCESS_AND_AUTH.md`; DDR §6

The deliverables table asks for Excel tabs for defects, model explanation and questions; the semantic solution and code as a ZIP; and a **Design Decision Record of at most two pages** covering options considered, choice, why, cost, and *what breaks at 100,000× the data volume*. Expected effort: **1–2 days**. Return within 7 days. Any LLM or AI coding assistant is allowed, with full disclosure. One sentence in the brief matters for the walkthrough: *"If you spend most of your time on infrastructure or tooling setup, you have misread the brief."*

> The scope question you will almost certainly get
>
> The repository goes well beyond 1–2 days: a two-factor login in the database, a web walkthrough, a deployment guide, a 55-slide deck, simulated layer agents. Do not deny it and do not apologise for it. The honest answer has three parts: (1) the three tasks were built and evaluated first; (2) the extras were added on explicit request and each one demonstrates a governance point rather than infrastructure (the second factor is enforced where the data is; onboarding is a data change); (3) the stack itself stayed minimal — one database, SQL files, plain Python, a Makefile — exactly so that the time went into the data, not tooling. If asked what you would cut: the deployment guide and the simulated agents first, then the web page (keep the console demo).

The way of working, as nine rules

These rules are the shortest honest answer to "how did you work?". Most design decisions follow from one of them.

1. **Executed, not asserted.** Every number in a deliverable is produced by a command whose run is logged (`dq/results/run_log.jsonl` → `STATUS_REPORT.md`). Claims carry a label: RUN, CODE, LLM, REC, ASSUMED. A step that did not run is printed `NOT EXECUTED`, never filled in.
2. **Land verbatim, never repair.** Bronze keeps the files as text with a SHA-256 per file. Silver converts, but every changed value keeps its original in a `_raw` column and a conversion note (D013, D014). A repaired value is indistinguishable from an observed one a week later.
3. **Grade, do not gate.** Staging filters nothing (D020). The rule layer decides afterwards, so the quality report describes the source, not what survived the loader.
4. **A severity must have a consequence.** `block` quarantines, `warn` flags and the flag travels to the answer, `info` logs. A severity without a mechanical effect is a comment.
5. **One definition, in one place.** "Problematic loan", "account owner", "incoming", "month-end balance" are decided once in the model and once in the semantic layer, and the duplication that remains is checked by the build (D006, D017).
6. **Distrust your own build.** The model is verified after every build (keys, grains, foreign keys, `core = stg - quarantine` on rows and money). Rules are proven by seeding 20 defects into a separate schema (D004). Security is tested by logging in as the restricted user and counting rows, not by reading the DDL (D010).
7. **The LLM is an untrusted query author.** It sees the semantic layer and the question, nothing else; its output is parsed and bounded; the filter sits where its text cannot reach (D009, D011).
8. **Evidence is kept apart.** Reference SQL and model output are never mixed; a second provider's run goes to separate files and never overwrites the recorded one.
9. **Write the decision down with its cost.** Every decision record has options, choice, reason and cost; every interpretation is an assumption with "what changes if wrong". Superseded decisions stay, marked (A36 → A39, D022 → D023).

> The one-line version
>
> "I built it so that every number can be traced to a command that produced it, nothing is silently repaired or dropped, and the AI is never trusted with anything it could get wrong quietly."

How AI tools were used

Full disclosure is a requirement of the brief, and the disclosure is in `AI_DISCLOSURE.md`. Be precise about three different roles:

Role	Tool	What it did
Builder	Claude Code (Claude Opus 5, later Opus 5.5 and Sonnet 5.5), interactive, under your direction	Wrote the repository: pipeline, SQL, rules, semantic layer, tests, documents — every step executed and checked against the database
LLM under test	Claude CLI non-interactive, **Claude Sonnet 5.5** (`claude-sonnet-5-5`)	Generated the SQL for the ten questions from the context pack only: no tools, no MCP, no filesystem, no database, working directory outside the repository, `num_turns: 1`

Role	Tool	What it did
Second LLM under test	Ollama, `qwen2.5-coder:3b`, local on the laptop CPU	Same questions through the same guard and login; results kept in `dq/results/local/`

"Hand written" reference SQL

`AI_DISCLOSURE.md` labels `sql/60_reference_queries.sql` "Hand written", and two documents say "a person worked out all ten answers by hand". The accurate statement is narrower: the reference SQL was **not generated by the model under test**; it was written in the same AI-assisted build session, independently of the evaluated run, and every reference answer was later **re-derived from the raw text tables with independent SQL** and matched exactly. Say it that way before anyone else does. (Lessons Learned, L-12.)

Why this stack

Choice	Alternatives	Why	Cost
PostgreSQL (16 on Linux/Docker, 18.6 on native Windows; identical DDL)	Cloud warehouse + dbt + orchestrator; DuckDB; SQLite	Task 3 is about *enforcement*. The only honest proof that a generated query cannot bypass a filter is to put the filter in the engine and connect as a restricted user. DuckDB has no row level security, so Task 3 would have become prose (D002, DDR §1)	Single node, no built-in lineage or scheduler; fine at this volume
Numbered SQL files + plain Python + Makefile (`scripts\windows.ps1` on Windows)	dbt, Airflow, Dagster	The brief warns against tooling; ordered SQL is readable in a review and each step is one command	Hand-rolled orchestration and checks
sqlglot for the SQL guard	Regular expressions; no guard	A parser counts and types statements; `SELECT/**/1; DROP TABLE x` is two statements to a parser and one line to a regex	A parser dependency; some legitimate SQL is refused (tables must be schema-qualified)
Claude CLI as a sandboxed completion (D005)	Anthropic API; local model only; an agent with database access	The exercise tests the *semantic layer*: an agent that could read tables would answer by looking. One interface (`src/llm_client.py`) keeps all providers swappable by one variable	A subprocess wrapper and a CLI dependency
Natural keys (D007)	Surrogate keys	One source, one extract, no history: surrogates would buy a join per dimension and longer LLM-written SQL	Revisit with history (SCD2)

2 The architecture in one picture

Six schemas in one PostgreSQL database are the spine. Each has one job and a contract with the next, and the security schema sits beside the model rather than on top of it.



There is a seventh schema, `inject`, used only by the test suite: a corrupted copy of staging with 20 seeded defects that proves the rules fire. It can never reach `core` (D004, D008).

What runs, in order

#	Command (`make …` or `scripts\windows.ps1 …`)	Builds	Code
1	`ingest`	`raw.*`, `raw.load_audit`	`src/ingest.py`, `sql/00_schemas.sql`
2	`profile`	`stg.*`, `dq/results/profile.md`, `candidates.md`	`sql/10_stg.sql`, `src/profile_data.py`
3	`dq`	`dq.results`, `dq.quarantine`, `dq.flags`, `dq.reconciliation`, `dq_report.md`, `defects.csv`	`src/dq_runner.py`, `dq/dq_rules.yaml`
4	`model`	`core.*`, model checks, `model_explanation.md`; then restores the `ai` views and policies (D012)	`sql/30_core.sql`, `src/build_model.py`
5	`ledger`	`docs/CLEANING_LEDGER.md`, measured from the database	`src/cleaning_ledger.py`
6	`reference`	`dq/results/reference_results.json`	`sql/60_reference_queries.sql`, `src/run_reference.py`
7	`semantic`	validation against the live schema; `semantic/context_pack.md`	`semantic/semantic_layer.yaml`, `src/semantic_context.py`
8	`entitlements`	`ai.*` views, roles, grants, 7 policies, demo principals, TOTP enrolment	`sql/40_ai_views.sql`, `sql/50_sec_rls.sql`, `sql/51_sec_auth.sql`, `src/build_security.py`
9	`questions`	`llm_logs/*.json`, `dq/results/llm_results.json`	`src/run_questions.py`, `src/llm_client.py`, `src/sql_guard.py`
10	`evaluate`	`dq/results/evaluation.json`	`src/evaluate.py`
11	`deliverables`	workbook, DDR (fails if over 2 pages), ZIP	`src/make_workbook.py`, `src/make_ddr.py`, `src/make_zip.py`
12	`status`	`STATUS_REPORT.md` from the run log	`src/make_status.py`

`make build` runs steps 1–8 plus the tests, without an LLM; `make all` runs everything.

One question, end to end

Take Q3, *"What percentage of our loans are problematic?"*, asked by a relationship manager.

1. **The question** is put into `semantic/prompt_template.md` together with the rendered semantic layer (`semantic/context_pack.md`). That text is the model's whole world: it has no tools, no files and no database.
2. **The model** returns one JSON object: `sql`, `reasoning`, `assumptions`, `caveats`. It reads in the pack that "problematic" is status B or D, already materialised as `is_problematic` on `ai.loan`, and that the denominator is all loans (A4).
3. **The guard** (`src/sql_guard.py`) parses the SQL with sqlglot: exactly one read-only SELECT, every table schema-qualified in `ai`, no forbidden or out-of-schema function, no CTE named like a table to smuggle one in, and a LIMIT forced to the 10,000-row cap.
4. **The login.** The SQL runs on a connection that authenticated as the user: database password, then a TOTP code verified *in the database* (`sec.verify_second_factor`). Until that succeeds, every entitlement resolves to nothing.
5. **Execution** happens inside `BEGIN READ ONLY` with a 60-second `statement_timeout`; at most cap + 1 rows are fetched.
6. **Row level security** on `core.fact_loan` (reached through the `security_invoker` view `ai.loan`) keeps only the loans of accounts this user is entitled to. The model never sees that predicate; it is not in the query, so it cannot be removed from the query.
7. **The answer** is the manager's own portfolio: `rm_north` sees 179 of the 682 loans. The same SQL run as `cro_global` (a portfolio holding every client) returns 76 of 682 = 11.14%, which is what the evaluation compares with the reference.
8. **Evidence.** The prompt, model, parameters and raw response go to `llm_logs/`, the result to `llm_results.json`, and `evaluate` compares values, not SQL text.

Where each kind of protection lives

"Three things stand between the question and the data and they do different jobs. The semantic layer removes the reasons to guess. The guard removes the ways to do harm with the *shape* of a query. The policy decides which rows exist for this user. Only the third is a security boundary; it would hold even if the first two failed."

Three contracts that hold the layers together

Contract	Checked by	What a failure would mean
raw = stg on row counts and money (11 checks)	DQ step, `dq.reconciliation`	Staging lost or invented rows or koruna
core = stg − quarantine on 7 row counts and 2 money sums	`build_model.reconcile_core` after every build	Cleaning leaked a row or an amount without a quarantine entry to explain it
The semantic layer matches the live schema — every column exists and is described, grains hold, join cardinality as declared, metrics name real columns, row counts current	`src/semantic_context.py` on every `make semantic`	The model would be given confident wrong information it cannot check

3 Step 1 — Acquire and ingest (Bronze)

Land the eight files exactly as they arrived, prove they arrived complete, and make every later value traceable to a file and a line.

COMMAND	CODE	OUTPUT
`make ingest` · `scripts\windows.ps1 ingest`	`src/ingest.py` , `sql/00_schemas.sql`	`raw.*` (8 tables, all text), `raw.load_audit`

What it does

- Reads the eight original `.asc` files (semicolon-separated, quoted headers; `trans.asc` is gzipped in the repository) from `data/raw/` . A bank-supplied copy dropped in `data/raw/provided/` would take precedence; none was received (A23).
- Loads **every field as text, unchanged** (C01: 1,079,680 rows) and stamps each row with `_source_file` and `_source_row` (C02).
- Writes a load audit per file: row count expected and loaded, and a **SHA-256** of the file, so a later reviewer can prove which bytes the numbers came from.
- Checks the counts against the published reference: account 4,500 · client 5,369 · disp 5,369 · card 892 · loan 682 · order 6,471 · trans 1,056,320 · district 77. All eight match.

Why it is built this way

Decision	Reason	Cost
Everything lands as text (C01)	A cast at load time destroys the evidence of the defect it trips over: a bad date would abort the load or vanish. Bronze is the one layer that cleans nothing	Typing is deferred to staging
Public mirror `github.com/jlacko/berka-dataset` (D003)	It ships the unmodified `.asc` originals and the original data description; its counts match the published figures exactly	Dependence on a mirror; mitigated by the hash and the count check
Public copy is primary; defects are injected into a **separate** schema (D004)	The public copy is clean enough that most rules would never fire, and a rule that has never fired proves nothing. Seeding defects into `inject` proves the rules while the ten answers stay on untouched data	The injector has to be kept in step with the rule families
Repository at the git root (D001)	`make all` from the clone root is what a reviewer tries first	—

Why bronze matters for an AI product

"When somebody disputes a number, the first question is what actually arrived. Bronze answers that with the file, its hash and the line number. Every later layer can be argued with; this one cannot."

Questions to expect

- *Why not type the columns on load?* Because the cast is where defects hide; staging casts with NULL-on-failure and keeps the original text.
- *How do you know the mirror is genuine?* Row counts match the published reference exactly for all eight files, and the files are the original `.asc` format, not a re-export.
- *Bronze has no dedicated tests* (said in the deck). The load audit and `RI_RAW_ROWCOUNT_MATCHES` are the checks; a bronze test module is listed as a next step.

4 Step 2 — Profile before writing rules

Measure every column first, write down what looks wrong with a count, and only then write the rules — so the rules come from the data and not from a checklist.

COMMAND	CODE	OUTPUT
`make profile`	`sql/10_stg.sql` , `src/profile_data.py`	`dq/results/profile.md` , `profile.json` , `candidates.md`

What it does

The profiler runs over the staging layer and reports, per column: type, nulls, null rate, distinct count, min, max and top values, including the lineage and `_raw` audit columns (a non-zero `_raw` count is itself a defect signal: it only fills when a cast changed or dropped a value). It then evaluates a list of **candidate defects** and writes them with counts to `dq/results/candidates.md` — *before* the rules exist. Each candidate is a hypothesis; the count and its note together decide the severity of the rule that follows.

Candidates present in the public copy (13)	Count
Partner bank and account missing together on postings	760,931
Three encodings of "no value" in the same column (`k_symbol`)	535,314
Structural NULL operation (interest postings)	183,114
Negative balance after a posting (overdraft)	2,999
Client district differs from the district of the account they own	409
Intra-day order undefined (accounts with several postings on their last date)	183
Running balance off by more than 1 CZK per account	136
Owner younger than 15 at account opening	114
Running balance off by more than 100 CZK per account	55
Amount is zero	14
Owner younger than 15 at loan start	6
District placeholder `?` in the 1995 columns (district 69, Jeseník)	1
Duplicate business key (same account, date, type, amount, balance)	1

Four candidates return zero on this copy and ship anyway, because a bank's own feed may carry them: a card held by a disponent, an account without transactions, a date outside the 1993–1998 window, an unparsable date or amount.

The balance candidates are not order independent

`candidates.md` calls the running-balance test "order independent". It is not quite: "the last reported balance" depends on which of several same-day postings is taken as last, and the profiler took the highest `trans_id` . That is how the tie-break problem of step 5 leaked into these counts (Lessons Learned, L-03).

5 Step 3 — Clean and transform (Silver)

Make the data usable without overwriting what the bank recorded: type it, normalise it, decode it, derive what the source leaves implicit — and filter nothing.

COMMAND
`make profile` (builds `stg`)

CODE
`sql/10_stg.sql` (functions `stg.norm_text`, `parse_numeric`, `parse_date`)

EVIDENCE
`docs/CLEANING_LEDGER.md`, measured from the database

The cleaning ledger, steps C03–C12

Every step is in the generated ledger with what it did, how much, where the original survives ("reversible") and which rule watches what it could not clean.

Step	What	Measured	Decision
C03	Trim, then map `''`, `?`, `NA`, `N/A`, `NULL`, `-`, `unknown` to NULL	1,380 rows	D014, A25
C04	Cast to numeric, returning NULL rather than raising	0 failed amounts	D013
C05	Parse `YYMMDD` into a real date; two-digit year read as 19xx	1,062,394 dates	D015, A17
C06	Strip the trailing clock time from `card.issued`	892	D015
C07	Decode `birth_number`: women carry month + 50	5,369 birth dates	D016, A18
C08	Derive gender from the same month offset	5,369	D016
C09	Decode every Czech code to an English label from one map	2,466,175 values, 28 codes	D017, A28
C10	Resolve the sign once: `PRIJEM` credit +, `VYDAJ` and `VYBER` debit −	1,056,320 postings	D018, A29
C11	Materialise "problematic" (status B or D) as a boolean	76 loans	A4
C12	Record every changed or dropped value in `_conversion_notes`	1,380 rows	D013

The decisions that matter most

Never repair in place (D013). Options were: repair bad values, drop defective rows, or transform without overwriting and let a rule layer decide. The third was chosen because a repaired value is indistinguishable from an observed one a week later, and dropping silently changes every denominator. Every changed value survives in a `_raw` column and a conversion note.

Placeholders become NULL, the original stays beside them (D014). Left alone, `?` counts as a value and every completeness number is wrong; replaced without a trace, the cleaning cannot be checked. The source spells "missing" seven ways; `k_symbol` alone arrives as a quoted empty string, an unquoted empty field and a single space.

Two-digit years are 19xx; a failed parse is NULL, not an error (D015). Safe because the dataset window is 1993–1998 and `VAL_DATE_OUT_OF_RANGE` fires on anything outside it. NULL on failure matters more than the century: a cast that throws aborts the load and hides exactly the defect a quality report needs to see.

Gender and birth date are derived, and labelled as derived (D016). Leaving `birth_number` packed would make every age question re-implement the decoding — and an LLM reading a context pack will get the month offset wrong. All 5,369 decode on this extract.

Codes and labels from one map, both kept (D017). `POPLATEK MESICNE` becomes `monthly`, but the code stays beside the label, and the build fails if a staged code has no label. The check earned its place: the order category `LEASING` (341 standing orders) had shipped unlabelled because the check did not cover that table (A28).

The sign is resolved once, in staging (D018). Every "incoming", "net flow" or "largest withdrawal" question turns on a `CASE` over Czech words. The documentation lists `PRIJEM` and `VYDAJ`; the data also has `VYBER`, which is a debit. A query that gets this wrong does not fail — it returns a confident, plausible, wrong number. So `signed_amount` and `direction` are computed once, and the running balance confirms the convention.

Graded, not gated (D020). `sql/10_stg.sql` states it as a contract: *nothing is filtered out here*. If staging dropped the duplicate posting, the uniqueness rule would have nothing to count and the report would say the data is clean.

The cleaning in one breath

"Twenty-one cleaning steps, and exactly one of them removes a row — the quarantine of one duplicate posting. Everything else converts without overwriting: the original value is one column away and the conversion is noted on the row. And I can prove nothing was lost: staging equals raw on every row count and every money sum."

Questions to expect

- *Isn't decoding gender from a birth number a business assumption?* Yes — A18, labelled as derived, and it is the only gender source in the data.
- *Why is `VYBER` a debit?* The data description only names `PRIJEM` / `VYDAJ`; the running balance decides it: under this convention the chain reconciles within 1 CZK for almost all accounts, under any other for almost none (A29).
- *What does `operation IS NULL` mean?* A bank-generated posting: all 183,114 such rows are interest credits (`UROK`). It is labelled "not applicable", not imputed (A27).
- *Is `is_problematic` defined in one place?* In meaning, yes; in code it is computed twice, in staging and again in core, with the same predicate (L-18).

6 Step 4 — Data quality rules and reconciliation

Grade every defect, give each grade a mechanical consequence, prove the rules actually fire, and prove that the cleaning lost nothing.

COMMAND	CODE	OUTPUT
`make dq`	`dq/dq_rules.yaml` (89 rules), `src/dq_runner.py`	`dq.results`, `dq.quarantine`, `dq.flags`, `dq_report.md`, `defects.csv`

The rule set

- **124 rules executed**: 89 written by hand in `dq/dq_rules.yaml` plus 35 generated by a per-column completeness sweep.
- Hand-written rules by dimension: validity 26, completeness 23, uniqueness 13, referential integrity 12, cross-table consistency 12, reconciliation 3. By severity: block 50, warn 23, info 16.
- Every rule carries a **dimension, a severity, a severity rationale written in terms of the questions it affects, a handling, and the questions affected**. The loader refuses a rule without them — a rule without reasoning "scores the same as no rule" in the brief's words.
- On the public copy **19 rules fire** (block 1, warn 6, info 12), and they describe **14 defect types** (block 1, warn 4, info 9) — several rules can describe one type, which is why the register has fewer rows than the report.

What each severity does

Severity	Consequence	Delivered run
BLOCK	Row quarantined with a reason naming the rule; excluded from core and from the ai views; reversible and countable	1 rule, **1 row**: `UNQ_TRANS_BUSINESS_KEY`
WARN	Row kept; flag written to `dq.flags`; flag travels onto the core row (`_dq_flags`) and a caveat with a row count goes into the semantic layer, so it reaches the answer	6 rules, **440 flags on 352 distinct rows**
INFO	Standardised or logged; no answer changes	12 rules

The best severity example

"Fourteen postings have amount zero. 'Amount must be positive' sounds obviously right, but those are real interest postings that accrued nothing. At block I would have quarantined valid rows and broken fourteen balance chains to fix nothing, so it is info. A *negative* amount would be block, because it means the sign was applied twice and money moves the wrong way. Same kind of check, opposite decision, and I can say why for each."

The 14 defect types

Defect type	Table	Dimension	Severity	Rows	Handling
Duplicate transaction by business key (distinct id, identical content)	trans	uniqueness	BLOCK	1	Keep the lowest trans_id, quarantine the rest with the reason naming the surviving row, so the decision is auditable rather than implicit.
Closing balance of the account's last day is not uniquely determined	trans	consistency	WARN	183	Tie-break documented in the semantic layer and applied identically in the reference SQL and in core.fact_account_month_end_balance; affected accounts flagged so the caveat can be quantified.
Running balance does not reconcile	trans	consistency	WARN	136	Account flagged; the month-end balance fact carries the residual so a user can see the size of the discrepancy next to the balance. | Account flagged as balance_unreconciled; Q9 reports the average together with the number of affected accounts and the total residual, so the reader can size the uncertainty instead of guessing at it.
Implausible age	client, loan	validity	WARN	120	Rows kept and flagged (_dq_flags carries 'owner_age_implausible'); the semantic layer attaches the caveat to the age metric so the LLM can repeat it. | Rows kept and flagged; Q7 reports the average with and without the flagged loans so the sensitivity is visible.
Counterparty missing on a transfer posting	trans	completeness	WARN	1	Row kept and flagged; counted separately from the structural NULLs.
Missing values in columns no certified metric reads	order, trans	completeness	INFO	1,073,386	Normalised to a real NULL in stg (placeholders '?', '', 'NA' and a bare space all collapse), original text preserved in the matching *_raw audit column, null rate published in dq/results/profile.md.
Counterparty bank and account absent for non-transfer postings	trans	completeness	INFO	760,931	Left as NULL by design; documented as structural in the semantic layer.
The same column encodes 'missing' three different ways in the source	trans	completeness	INFO	535,314	Collapsed to a single NULL by stg.norm_text; the raw spelling is kept in k_symbol_raw. Rule runs against raw, so it keeps reporting even after stg is clean.
Transaction operation absent (structurally, for interest postings)	trans	completeness	INFO	183,114	Left as NULL by design, labelled 'not applicable (bank-generated posting)' in the semantic layer so an LLM does not treat it as missing.
Balance after the transaction is negative (overdraft)	trans	validity	INFO	2,999	Logged only; balances are never clipped.
Owner's own district differs from the district of the account they own	disp	consistency	INFO	409	Logged with its rate. Q1 uses the client's own district, Q5 uses the account's district; both are stated in the semantic layer and in ASSUMPTIONS so an LLM cannot pick silently.
Two clients share birth number and district (possible duplicate person)	client	uniqueness	INFO	26	Logged only; no row is changed or removed.
Transaction amount is zero or negative	trans	validity	INFO	14	Logged only; the postings stay in core with amount = 0.

Defect type	Table	Dimension	Severity	Rows	Handling
Placeholder '?' in the 1995 unemployment and crime columns	district	completeness	INFO	1	'?' normalised to NULL in stg, original text kept in unemployment_rate_1995_raw / crimes_1995_raw, district row otherwise used normally.

The "rows" column counts what each rule reports. For the two completeness types it counts the same missing `k_symbol` row once per column it is missing from (code and label), so it is a count of missing values, not of distinct rows.

The firing warn rules, and what they mean for the answers

Rule	Rows	Affects	Meaning
`CON_INTRADAY_ORDER_AMBIGUOUS`	183	Q9	Several postings on an account's last date; the source has no time of day
`VAL_OWNER_AGE_AT_ACCOUNT_OPEN`	114	Q7, Q8	Owner younger than 15 when the account opened (minimum 10.3 years)
`CON_BALANCE_RECONCILIATION_ROUNDING`	81	Q9	Balance chain off by 1–100 CZK (amounts carry one decimal)
`CON_BALANCE_RECONCILIATION_MATERIAL`	55	Q9	Balance chain off by more than 100 CZK
`VAL_OWNER_AGE_AT_LOAN_START`	6	Q7	Owner younger than 15 at loan start — a minor cannot take a loan
`COMP_TRANS_PARTNER_ON_TRANSFER`	1	—	Counterparty missing on a transfer posting

Two of these counts are weaker than they look

- 41 of the 55 "material" balance breaks — including the account said to be "off by exactly 1,200" — are artefacts of the month-end tie-break, not of the source (Lessons Learned L-02, L-03). Corrected after delivery: 14.
- The only **block** is a duplicate whose amount is 0.00 CZK. The rule requires the same resulting balance, so by construction it can only catch duplicates that did not move the balance — zero-amount ones. Meanwhile 1,081 groups of identical repeated interest postings (157,289.70 CZK) are not flagged at all; whether they are duplicates is a business question (L-04).

Proving the rules fire: the defect injector

A rule that never fires on clean data proves nothing. `sql/70_inject_defects.sql` builds a corrupted copy of staging in schema `inject` with **20 seeded defects, one per rule family**, deterministically. `tests/test_dq.py` asserts each one is caught by the rule meant to catch it. The ten answers are never computed on that copy (D004).

The injector produced the most useful bug of the project (D008): `dq.v_blocked` was not scoped by schema, so running the rule tests made the *injected* defects exclude *real* rows — 76 districts instead of 77, 11,024 transactions orphaned. The model checks failed before any deliverable was written. Fix: the views filter `target_schema = 'stg'`, and a comment says why the filter is load-bearing. The lesson: verify the model, do not trust a green build.

Reconciliation: proving nothing was lost

Check	Where	Result
raw = stg: 8 row counts and 3 money sums (transactions, loans, standing orders)	`dq.reconciliation`, DQ step	11 of 11 balanced
core = stg − quarantine: 7 row counts (trans 1,056,320 − 1 = 1,056,319) and 2 money sums (transactions 6,257,793,560.30; loans 103,261,740.00)	`build_model.reconcile_core`, model step	9 of 9 balanced, every delta 0
Per-account balance chain (sum of postings vs last reported balance)	rules `CON_BALANCE_RECONCILIATION_*`	Reported, not corrected (A20, A30)

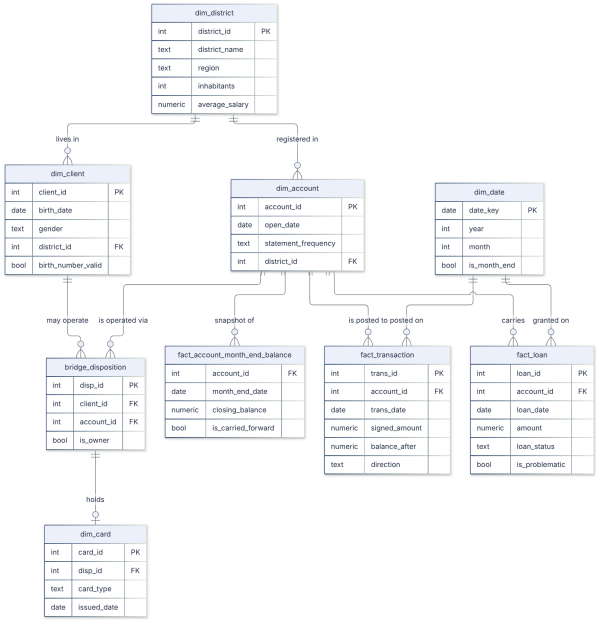
"Would the money reconciliation catch the quarantine?"

No: the quarantined posting is 0.00 CZK, so the money sums are identical either way. The row count catches it (1,056,320 − 1).

7 Step 5 — The model (Gold)

A star schema with stated grains, one definition per business term, the data quality flags attached to the rows they concern, and checks after every build.

COMMAND	CODE	OUTPUT
`make model`	`sql/30_core.sql` , `src/build_model.py`	`core.*` , `deliverables/model_explanation.md`



The core star schema (VISUAL_GUIDE figure 4). The disposition bridge resolves the many-to-many between clients and accounts and carries `is_owner`.

The objects and their grains

Object	Kind	Grain (one row per)	Rows	Why it exists
`dim_client`	dimension	client	5,369	Person; derived birth date and gender, labelled derived
`dim_account`	dimension	account	4,500	Open date, statement frequency, account district
`dim_district`	dimension	district	77	Region and demographics
`dim_date`	dimension	calendar day	2,191	Generated for 1993-01-01..1998-12-31 (C19); lets "per month" queries show zeros
`dim_card`	dimension	card	892	Card type, issue date, via the holder's disposition
`bridge_disposition`	bridge	disposition	5,369	Who may operate which account; `is_owner`
`fact_transaction`	fact	posting	1,056,319	Signed amount, direction, balance after (quarantine removed)
`fact_loan`	fact	loan	682	Amount, status, `is_problematic`
`fact_account_month_end_balance`	derived fact	account × month end	185,615	The Q9 balance, decided once (558 carried forward)
`v_account_owner` , `v_loan_enriched` , `v_client_region`	curated views	account / loan / client	—	The single places that decide "owner", the loan's owner attributes, and the client's region
`ref_codes`	reference	code	28	The one code-to-label map (D006, D017)

Why a star with a bridge

Options: one wide table at posting grain; Data Vault; the source model as-is. **Choice:** four dimensions from the source tables (client, account, district, card) plus a generated calendar, a bridge, two base facts and a derived fact. (The documents say "four dimensions"; someone counting `dim_` tables will find five — the fifth is the generated `dim_date` .) **Why:** the ten questions live at four grains (client, account, loan, posting). A wide table turns every count into `count(distinct …)` — exactly the construct generated SQL gets wrong quietly. Data Vault suits many sources with history; there is one source and one extract. **Cost:** joins, each declared in the semantic layer with a measured cardinality, and a derived fact to rebuild. Natural keys, not surrogates (D007): one source, no history, and shorter SQL for the model.

One definition, decided here

Term	Decided in	Definition
Account owner	`core.v_account_owner`	The client on the account's `OWNER` disposition; exactly one per account (block rule `UNQ_ACCOUNT_SINGLE_OWNER` , 4,500 owners for 4,500 accounts)

Term	Decided in	Definition
Problematic loan	`fact_loan.is_problematic`	Status B (finished, unpaid) or D (running, in debt): 76 of 682 (A4)
Incoming money	`fact_transaction.direction = 'credit'`	Type `PRIJEM`, summed on the unsigned amount (A7)
Month-end balance	`fact_account_month_end_balance`	Balance after the last posting on or before the month end; carried forward when the month had none (A14, A15, D019)
Owner's age at loan start	`v_loan_enriched`	Exact fractional years from the decoded birth date (A10)

The month-end balance, and its known limitation

D019 builds one row per account per month end from the account's first posting month to 1998-12: the balance after the latest posting in that month; a month with no posting carries the previous balance forward, with `is_carried_forward` and `balance_as_of_month_end` exposed (558 such rows). Options rejected: compute it on the fly per query (two answers to one question guaranteed), or skip quiet months (biases the average upward).

The limitation. When an account has several postings on its last day, the delivered rule picks the one with the **highest `trans_id`** — but the source records no time of day and `trans_id` is not chronological. Interest postings carry high ids but are booked before the month's statement fee, so the rule often reads the balance *before* the fee. This was disclosed at delivery (A14, caveat C2, DDR §3, delivery audit item 3) as a known limitation: Q9's average barely moves; individual balances do.

> What changed after delivery (D028)
>
> The running balance *does* record the order: each posting starts at `balance_after - signed_amount`, so the postings of a day form a chain and its end is the closing posting. Over 185,057 posted account-months the chain resolves 98.8%; the highest id picked the right posting only 0.4% of the time. Q9 moves from 43,808.94 to 43,810.47; 107 accounts close differently at 1998-12-31, by up to 1,200.00 CZK. Lessons Learned L-01 has the full story.

Verified after every build

`build_model` checks primary keys, foreign keys, declared grains (one row per key), code labels against `ref_codes`, and `core = stg - quarantine`; the delivery run reported 0 key, foreign key, grain or code problems and 9 balanced reconciliation checks. Rebuilding core drops the `ai` views that depend on it, so `make model` restores the views, policies and grants in the same command (D012) — a lesson learned when a single re-run left the AI surface missing.

8 Step 6 — Reference answers, the marking scheme

Before any model is asked anything, write the ten answers as SQL against the core model, state the interpretation each one implements, and keep them away from the model.

COMMAND	CODE	OUTPUT
`make reference`	`sql/60_reference_queries.sql` , `src/run_reference.py`	`dq/results/reference_results.json`

- One query per question, each preceded by its **interpretation** (A1–A16): which district for "region", what "problematic" means, which date "granted" uses, how the owner is found, how age is computed, what the balance at a date is.
- Where a question can reasonably be read another way, the alternative and its size are written next to the query and in `ASSUMPTIONS.md` — so a reviewer who prefers the other reading can see exactly what changes.
- The reference SQL, the reference figures and the evaluation criteria are **never in the model's context**; `tests/test_semantic.py::test_context_pack_contains_no_answers` asserts that no reference figure leaks into the pack.
- At delivery, all ten reference queries were re-executed against the live database and were identical to the stored results. Separately, every reference answer was **re-derived from the raw text tables with independent SQL** (no `stg` , no `core`) and matched exactly.

Who wrote the reference SQL

It was written in the AI-assisted build session, not by the model under test, and not literally by hand. That is enough to keep the evaluation meaningful — the model never saw it — but the words "hand written" and "a person worked it out" in some documents overstate it. The independent re-derivation from raw is the stronger argument; use it (L-12).

Why write the answers first

"An evaluation is only as good as its marking scheme. I wrote the reference answers with their interpretations before the model was asked anything, kept them out of its context, and then rebuilt each one from the raw text with separate SQL to make sure the model and the reference were not wrong in the same way."

9 Step 7 — The semantic layer

One YAML file holds every business meaning the model is allowed to use, and the build refuses it if it disagrees with the live database.

COMMAND	CODE	OUTPUT
`make semantic`	`semantic/semantic_layer.yaml`, `src/semantic_context.py`, `semantic/prompt_template.md`	`semantic/context_pack.md` (33,662 characters at delivery)

What is in the file

Section	Content at delivery
`meta`	Dialect (PostgreSQL), query schema `ai`, currency CZK, date range 1993-01-01..1998-12-31
`query_rules`	12 rules, QR1–QR12 (below)
`glossary`	10 terms, each with what it **is** and what it is **not**: account owner, disponent, problematic loan, incoming transaction, new account, region, statement frequency, month-end balance, quarter, age at loan start
`metrics`	12 certified metrics with their entity, expression, time column: `client_count`, `account_count`, `loan_count`, `total_loan_amount`, `average_loan_amount`, `problematic_loan_share`, `incoming_transaction_amount`, `outgoing_transaction_amount`, `net_flow`, `average_month_end_balance`, `average_age_at_loan_start`, `gender_share`
`entities`	11 `ai` views with grain, primary key, every column described, allowed values, 19 joins with cardinality and optionality
`caveats`	7 caveats C1–C7, each tied to a rule, with a severity and a row count
`entitlements`	Tells the model that a filter exists inside the database, that it must not add or remove one, and what a restricted answer means
`question_patterns`	Worked patterns for common question shapes

The query rules the model is given

Rule	Instruction
QR1	Read only from `ai`; never reference `core`, `stg`, `raw`, `dq`, `sec` or `inject`
QR2	Exactly one statement, a single SELECT (a leading WITH is fine)
QR3	No DDL or DML — no INSERT, UPDATE, DELETE, CREATE, ALTER, DROP, GRANT, SET, COPY, CALL
QR4	Do not try to widen what you can see; do not mention entitlements in the SQL
QR5	Use the certified metric definitions verbatim; never re-derive `is_problematic`, `direction` or an age
QR6	For "per month / quarter / year", join `ai.calendar` so empty periods show as zero
QR7	Prefer the pre-joined entity (`ai.loan`, `ai.account_owner` carry owner, card type, frequency)
QR8	Return supporting counts next to a percentage or an average; surface the relevant caveat
QR9	Round percentages to two decimals; do not round monetary sums
QR10	If the question cannot be answered from the entities, say what is missing instead of guessing a column
QR11	Filter time on the metric's own date column with half-open ranges; Q1–Q4 in a user's question are calendar quarters
QR12	Count and sum at the grain of the entity asked about; never aggregate after joining to the "many" side

QR11 and QR12 came from the semantic model audit (2026-10-28), together with the `quarter` glossary entry, a `time_column` on every metric and measured cardinality on every join.

Validated against the live database

`src/semantic_context.py` checks the YAML against `information_schema` and the data on every build. Any of these **fails the build**:

- a described column that does not exist, or an existing column that is not described;
- a declared grain that is not one row per key;
- a join whose measured cardinality or optionality differs from the declared one;
- a stale row count;
- a metric whose entity is not described, or whose expression, filter or grouping names a column that does not exist (added at delivery — `gender_share` had said "any entity with a gender column").

At delivery: 0 problems; 11 entities, 105 columns, 19 joins, 12 metrics, 7 caveats.

> Why a validated file instead of prompt text
>
> "A semantic layer that has drifted from the schema is worse than none: it gives the model confident, wrong information that the model has no way to check. So the layer is a build artefact. If someone renames a column or a grain stops holding, the build fails before any answer is produced."

What makes the layer AI-ready, specifically

- **Business names and decoded labels** on every `ai` column; codes kept beside labels.
- **Pre-joined views**: `ai.loan` carries the owner, the owner's card type, exact age at loan start and the account's statement frequency, so Q2, Q3, Q4, Q7 and Q10 need no joins.
- **"Is not" definitions** in the glossary: *a disponent is not the owner*; *incoming is not a net flow*; *a quarter is not a reference question*.
- **Caveats with counts** that the model is told to surface (QR8), so the warn flags of step 4 reach the person reading the answer.
- **A refusal path** (QR10): the model is told to say what is missing rather than invent a column.

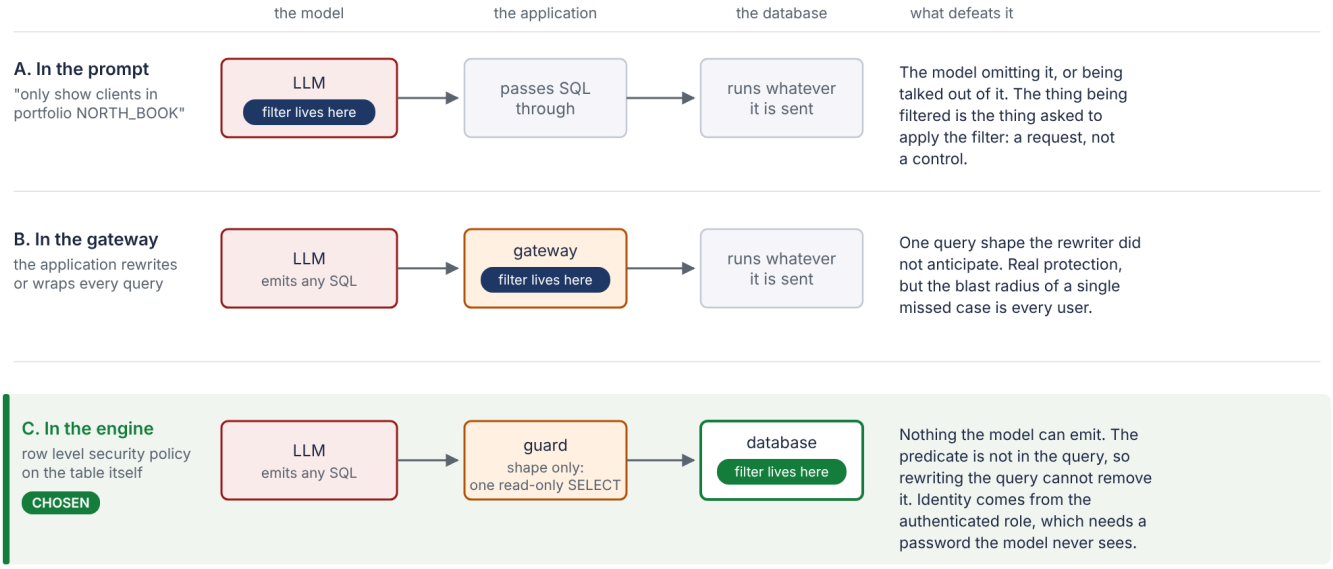
Questions to expect

- *Doesn't pre-joining for the questions over-fit?* Partly. The pack even says which entity "answers Q2, Q3, Q4, Q7 and Q10". It shows the model where to look without giving numbers; a hold-out set of unseen questions is the honest remedy (L-08).
- *What stops the model from using a column that is not there?* Before delivery: only the database, which raises an error the runner records — a safe failure, but a failure. The observed case was `T2.gender` on `ai.account_owner` (the column is `owner_gender`). After delivery, the intent verifier rejects it before execution (D029, L-07).
- *Is the 1,200 CZK in caveat C1 right?* No — it is an artefact of the month-end tie-break, and it is in the text the model reads (L-02).

10 Step 8 — Entitlements: where the filter sits

Task 3 asked for an explanation. It was built and tested instead: the filter is a row level security policy on the base tables, the identity is the authenticated database role, and access is data, so onboarding needs no code.

COMMAND	CODE	EVIDENCE
`make entitlements`	`sql/40_ai_views.sql` , `sql/50_sec_rls.sql` , `sql/51_sec_auth.sql` , `src/build_security.py`	`tests/test_rls.py` , workbook tab *Entitlements*, `docs/ACCESS_AND_AUTH.md`



Three places a filter can live (VISUAL_GUIDE figure 5). A prompt instruction is a request; a gateway is defeated by one query shape it did not anticipate; a policy in the engine holds whatever SQL arrives.

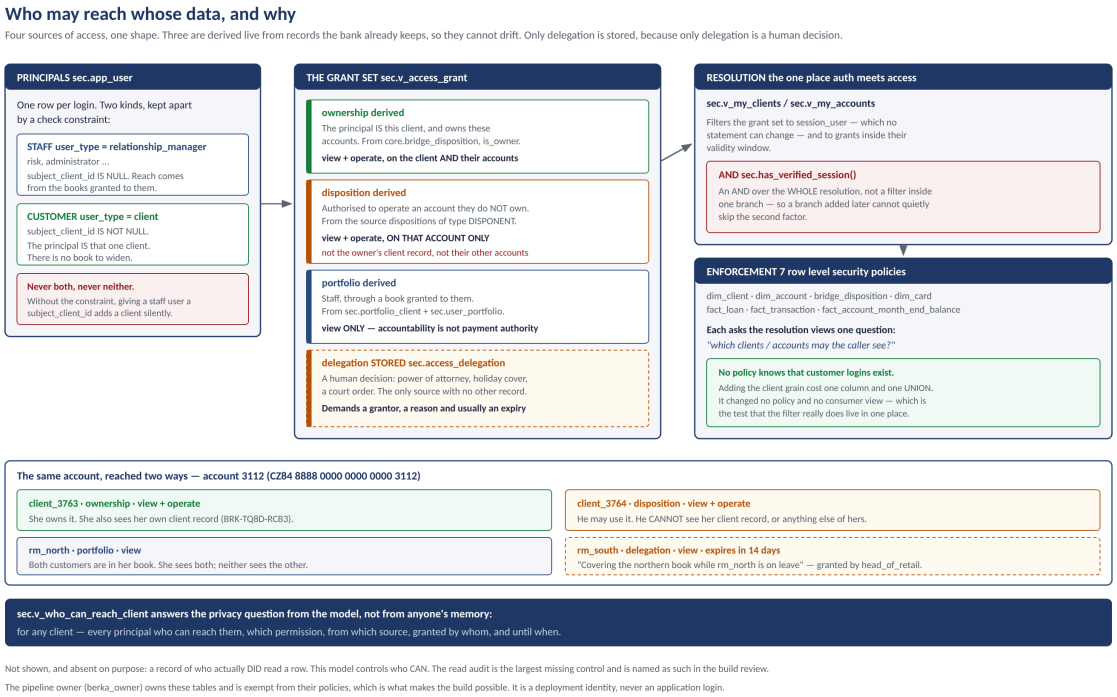
The three answers the brief asked for

Where the filter sits. Seven row level security policies on the seven client-bearing core tables: `dim_client` , `dim_account` , `bridge_disposition` , `dim_card` , `fact_loan` , `fact_transaction` , `fact_account_month_end_balance` . Every `ai` view, and every `core` view between a consumer and a policy, is `security_invoker` , so the policy is evaluated as the caller, not as the view's owner (D010). Reference data (districts, calendar, code list) is deliberately unfiltered: it holds no client data and filtering it would break every group-by (A22, A37).

Why an LLM-generated query cannot bypass it. The model produces text. The predicate is not in that text, so no rewrite of the text removes it; the policy sits inside the plan of every statement on the connection. The identity the policy reads is `session_user` — the database role that logged in — which no SQL statement can change without a password the model never sees (D009). A session variable such as `app.end_user` was rejected exactly because any statement can set one. No consumer role has `BYPASSRLS` or superuser; the bank-wide risk user `cro_global` is *a portfolio containing every client*, not an exemption (`rolbypassrls = false`).

How a new user or portfolio is onboarded without code. Insert the principal into `sec.app_user` , attach a portfolio in `sec.user_portfolio` (or put clients into a portfolio in `sec.portfolio_client`), create the login role with its credential, enrol the second factor. Grants are *derived live* from those rows, so the new user's access works immediately. Offboarding is one `UPDATE` that deactivates the principal. No policy, view or query changes.

Four sources of access, one shape



The access model (VISUAL_GUIDE figure 11). Note: the figure says the disponent "cannot see her client record, or anything else of hers"; the client record is right, "anything else" is not — see the weak spot below.

Source	Derived from	Grants	Decision
Ownership	`core.bridge_disposition` with `is_owner`	view + operate on the client and the accounts they own	D021
Disposition	dispositions of type DISPONENT	view + operate on **that account only** — never the owner's client record or other accounts	D023, D024, A39, A40
Portfolio	`sec.portfolio_client` + `sec.user_portfolio`	view only on the clients of the book (accountability is not payment authority)	D009, A41
Delegation	`sec.access_delegation` — the only *stored* grant	what a human decided, with grantor, reason and expiry (holiday cover, power of attorney)	D027

All four flow into `sec.v_access_grant`; `sec.v_my_clients` / `sec.v_my_accounts` filter it to `session_user` and to grants inside their validity window, **AND** `sec.has_verified_session()` over the whole resolution, so a branch added later cannot skip the second factor. Two kinds of principal share one policy set: staff, entitled to the union of their books, and self-service customers, entitled to exactly themselves (a check constraint forbids "both" and "neither"). Adding the customer grain cost one column and one UNION and changed no policy — the test that the filter really lives in one place.

D027 is a lesson worth telling: the grant set was first *materialised*, and granting a portfolio inserted a membership row but no grant, so the new access did nothing until the next rebuild while the tables said it should work. Deriving the grants live removed the second description that could drift.

Measured, by logging in as each role

Principal	Clients	Accounts	Loans	Transactions
`cro_global` (portfolio of every client)	5,369	4,500	682	1,056,319
`rm_north`	1,481	1,241	179	294,622
`rm_south` (1,386 in its book + 1 delegated client)	1,387	1,158	195	272,770
`rm_unassigned` (no portfolio)	**0**	0	0	0
`client_3764` (a disponent: one account, not its owner)	1	1	1	439

A brand-new user with no portfolio sees **nothing**, not everything — the failure mode of a badly built filter is the opposite.

Tested by attacking it

`tests/test_rls.py` connects as restricted users and tries to get out: setting a session variable to another user's name, `SET ROLE`, reading `core` directly instead of through the views, reaching `raw` / `stg`, granting itself access, dropping or disabling a policy, writing, widening the result through a join — nine bypass attempts, all refused. It also checks that two customers of the *same* relationship manager cannot see each other (13 targeted attempts including going around the views), the disponent and delegation cases, onboarding and deactivation as pure data changes, that every protected table has RLS enabled, and that every view between consumer and policy is `security_invoker`.

The security bug worth telling: before `security_invoker` was set on three curated `core` views, `ai.loan` returned all 682 loans to a user entitled to none, while `core.fact_loan` returned 0 in the same session. The design was right and the DDL was one setting short; it was found only because the test counted rows as the restricted user (D010).

Privacy claims that go too far (verified 2026-11-04)

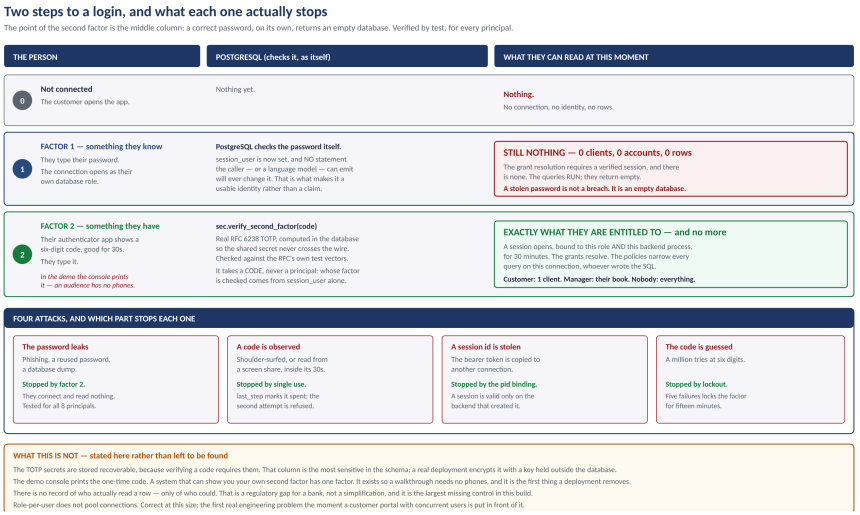
- **A disponent sees more of the owner than the documents say.** Logged in as `client_3764` (disponent on account 3112, owned by client 3763): `ai.client` correctly returns only 3764, but `ai.card` returns the owner's card (card 587, junior, issued 1996-08-05, `holder_client_id` 3763) and `core.bridge_disposition` returns the owner's disposition row. The card and disposition policies key on the account. (L-05)
- **Hidden data becomes false data.** For the same user, `ai.loan` shows the loan with `owner_card_type = 'no card'` and a NULL statement frequency — the truth is *junior* and *monthly*. The owner join returns nothing under RLS and the view's `coalesce(…, 'no card')` turns "not visible" into a wrong value. Q4 or Q10 asked by a disponent would be wrong, not merely restricted. (L-06)
- `sec.v_who_can_reach_client` has a latent bug: a `<> (SELECT … LIMIT 1)` branch returns NULL for a client without a login, silently dropping indirect grants; `u.db_role <> ''` is a no-op. Harmless with today's seed. (L-11)
- Not built: a read audit ("who *did* see"), field masking, connection pooling (a role per user conflicts with naive pooling — answer: a pool per role, or a trusted gateway doing `SET ROLE`).

Where the filter sits, in three sentences

"The filter is a row level security policy on the base tables, evaluated as the caller because every view in between is security-invoker. The caller is the database role that authenticated, which no SQL can change, so the model's text has nothing to remove. And access is rows in `sec`, derived live, so onboarding is an insert and offboarding is an update."

11 Step 9 — Authentication: two factors, in the database

A password alone reaches nothing. The second factor is verified by the database itself, so no application in front of it can skip it.



The two-step login (VISUAL_GUIDE figure 12).

How it works

1. **Factor 1** is the database password of the principal's own login role.
2. **Factor 2** is a time-based one-time code (TOTP, RFC 6238), implemented in SQL. The client calls `sec.verify_second_factor(code)`. The function takes *only a code*, never a principal: it uses `session_user`, so nobody can verify on someone else's behalf.
3. A correct code is **single use** (the last used time step is stored). Five committed failures lock the factor for 15 minutes; while locked, even the right code is refused.
4. Success opens a session in `sec.auth_session`, bound to the database role and the backend process of that connection, for 30 minutes.
5. `sec.has_verified_session()` is ANDed into the entitlement resolution. Without it every policy resolves to the empty set: **password only → 0 rows**.

Identifiers (D025): `client_id` for joins; `client_ref` (`BRK-XXXX-XXXX`, an HMAC under a secret salt) to quote without enabling enumeration; `account_ref`, an IBAN with real mod-97 check digits and a fictional bank code, for the number a customer recognises.

Why in the database (D026)

"Most systems enforce the second factor in the application, which is a defensible architecture — but it makes the control a property of the application rather than of the data, the same mistake I rejected for entitlements. Here the claim is testable in the only way that means anything: hold the correct password, present no code, and read nothing."

Limits to state before you are asked

- **Rollback gap (found in the delivery audit).** The failure counter is written inside the caller's transaction. The project's own client commits every attempt, but someone who holds a principal's password and connects directly can roll each failed guess back and is never locked out. Recorded as a strict expected failure in `tests/test_auth.py`; fix: count attempts outside the caller's transaction, or accept logins only through a gateway. RLS itself is not affected: no factor, no rows. (L-09)
- **Session bound to the backend PID only.** `backend_start` is stored but never compared, so a reused PID for the same role within 30 minutes could inherit a session (A45, L-10).
- **TOTP secrets are stored recoverable** in `sec.auth_factor` (A44) — verification needs the secret; production would encrypt them with a key outside the database.
- **TOTP is phishable in real time** (A43); production should prefer WebAuthn/passkeys.
- **The demo shows its own codes** (`/api/code`, console) and personas share a demo password — a demo affordance, labelled as such, the first thing a deployment removes (A46).

12 Step 10 — The LLM as a dumb and dangerous client

Design the AI path as if the assistant were a junior analyst with production credentials and no judgement: it will guess at anything the data does not spell out, and whatever it writes will run against customer data.

The threat model

"Dumb" — it will be wrong without knowing	"Dangerous" — it writes code that runs
Invents a column that sounds right (`gender` on a table that has `owner_gender`)	Emits DDL or DML (`DROP` , `UPDATE`)
Uses the wrong date (account open date for a loan question)	Smuggles a second statement past a filter (`SELECT/**/1; DELETE …`)
Joins to the many side and inflates a sum (loans × postings)	Reads internal schemas whose columns mean something else (`stg` , `raw`)
Counts instead of computing a share; ranks the wrong measure	Calls functions that read settings or files (`current_setting` , out-of-schema functions)
Confuses "Q1" the quarter with "Q1" the reference question	Returns millions of rows or runs for hours
Silently re-derives a definition (problematic, age, sign)	Tries to widen its own entitlement or impersonate another user

The defence, layer by layer

Layer	What it enforces	What it does not do
Isolation	Claude CLI with no MCP servers, every tool disallowed, explicit system prompt, working directory outside the repository; one turn, one JSON object. The model never sees the database, the reference SQL or the evaluation	Make the model right
Semantic layer	Removes the reasons to guess: certified metrics, "is not" glossary, grains, join cardinality, caveats with counts, 12 query rules, a refusal path (QR10)	Check that the model followed it
SQL guard	sqlglot parse: one statement; SELECT only; every table schema-qualified in `ai` ; a CTE named like a table cannot stand in for it; none of the blocklisted functions (`pg_read_file` , `pg_sleep` , `set_config` , `dblink` …); a blocklist, so `current_setting('data_directory')` passes and PostgreSQL refuses it (LOW-1); LIMIT added or tightened to the 10,000-row cap	**Resolve columns or judge meaning**
Execution envelope	`BEGIN READ ONLY` ; `SET LOCAL statement_timeout` 60 s; at most cap + 1 rows fetched; runs on the persona's own two-factor connection; SELECT-only grants	—
Row level security	Decides which rows exist for this user, whatever the SQL says	—
PostgreSQL itself	A name the schema does not have is an error, returned and recorded — never answered	Catch a wrong-but-valid query
Evaluation	Value-based comparison with independent reference SQL, after the fact	Protect a live user

The sentence to never say

"The query passed the guard, so it is correct." The guard proves a query is *safe to run*; it says nothing about whether it *answers the question*. In the delivered version the only checks on meaning are the database rejecting unknown names and the offline evaluation. The intent checker (`scripts/ir_check_prototype.py`) is a prototype and is not imported by the application.

The observed failure that proves the point

Asked through the application: *"Find male or female loans in Q1 1995."* The model wrote:

```
SELECT COUNT(*) AS total_loans,
       SUM(CASE WHEN T2.gender = 'male' THEN 1 ELSE 0 END) AS male_loans,
       SUM(CASE WHEN T2.gender = 'female' THEN 1 ELSE 0 END) AS female_loans
FROM ai.loan AS T1 INNER JOIN ai.account_owner AS T2 ON T1.account_id = T2.account_id
WHERE T1.loan_date ⩾ '1995-01-01' AND T1.loan_date < '1995-04-01'
```

The guard allowed it — it is one safe SELECT on `ai` . PostgreSQL then raised `column t2.gender does not exist` . Gender lives on `ai.client` ; `ai.loan` and `ai.account_owner` expose it as `owner_gender` . The root cause was in the semantic layer: the `gender_share` metric said "any entity with a gender column". The delivery fixed the metric text and made the validator refuse a metric that names a missing column; it did not add meaning verification. The failure was **safe** — no rows, an error recorded — but it was a failure.

After delivery (D029)

The model now returns an intent (IR) next to its SQL — metric, entity, counting grain, half-open periods, grouping, filters — and `src/sql_ir.py` checks both before any database session is opened: every column resolved against the semantic layer, the period filter evaluated day by day, joins declared and not fanning out, grouping and filters as declared. Then EXPLAIN, guarded execution under RLS, and the certified metric recomputed from the IR; rows are shown only when both agree. The observed SQL is refused naming `owner_gender` , and the corrected answer is 28 loans: 18 male-owned, 10 female-owned (A47, pending business confirmation).

The local model shows what "dumb" looks like

The optional local model (Ollama `qwen2.5-coder:3b`, CPU, 2026-10-27, older pack) scored 6 match, 2 partial, 2 mismatch. Every miss was safe SQL that the guard correctly let through:

Q	What the small model did	Why it is wrong
Q10	`count(*) FILTER (WHERE is_problematic)` per frequency, `ORDER BY … LIMIT 1` → *monthly*	A count, not a share — exactly the trap the glossary warns about. The share answer is *after transaction*, 18.75%
Q6	Joined `ai.account.open_date = ai.calendar.month_start` and grouped by month	Matches only accounts opened on the 1st, then fans out across the month's days: 279 "new accounts" in January instead of 106
Q7	Grouped by age *and* status, counted rows	Never averaged the age
Q3	Returned 11.14% without the supporting counts	Right number; partial only because the evaluator also expects 76 and 682 (QR8)

What the two runs together show

"A strong model with a good semantic layer got all ten right; a small local model made exactly the mistakes the semantic layer warns about — share versus count, a fan-out join. Every one of those wrong queries was perfectly safe, which is the point: governance has to make wrong answers detectable, not only dangerous queries impossible."

13 Step 11 — Asking the questions and evaluating

COMMANDS
`make questions` , `make evaluate` ; local: `questions-local` , `evaluate-local`

CODE
`src/run_questions.py` , `src/llm_client.py` , `src/evaluate.py`

OUTPUT
`llm_logs/` , `dq/results/llm_results.json` , `evaluation.json` ; local run under `dq/results/local/`

The runner

For each question: build the prompt from the template and the rendered pack; call the model; parse its JSON (tolerant of a markdown fence, not of a missing `sql`); guard the SQL; log in as `cro_global` with both factors; execute under the envelope; record everything. **One retry** is allowed when the JSON did not parse or the guard or database refused, with the reason fed back — what a real text-to-SQL service does. Honesty rules enforced in code: no SQL is ever hand-corrected; a failed call is `NOT EXECUTED` with an empty SQL cell; the result is whatever the database returned, including an error. Every call — prompt, model, parameters, raw response, timing, cost — is written to `llm_logs/` .

Why `cro_global` ? It holds a portfolio of every client, so its answers are comparable with the reference, while still passing through the policies like any user. The same SQL as `rm_north` returns that manager's slice.

The evaluation

Value-based, not text-based (D011). Two correct queries name their columns differently and may return extra columns; comparing SQL or column names would measure style. For each question the evaluator extracts the reference's (dimension → measure) pairs and searches the model's rows for each one: a matching dimension cell and, on the same row, a number equal within an **absolute tolerance of 0.011** (enough for two-decimal rounding, not for a wrong number). Row order is checked for the three ranking questions (Q5, Q6, Q10). Verdicts: match, partial, mismatch, not executed.

Two evaluator lessons, both disclosed: a relative tolerance of 1e-4 (the first version) would have accepted the Q5 top district wrong by 11,513 CZK; and Q6 once scored `mismatch` although the counts were identical, because the model wrote `1996-01` where the evaluator looked for `1` . The fix was to accept any published representation of a dimension — and that change was made after seeing an answer, which is why it is written down in D011.

The results

Run	Provider / model	Date, commit, pack	Result
Recorded (submission evidence)	`claude_cli` / Claude Sonnet 5.5	2026-10-25, `a7da25a` , 29,500-character pack	10 answered on the first attempt; **10 of 10 match**; about 2 minutes and USD 0.40
Local (separate files)	`ollama` / qwen2.5-coder:3b	2026-10-27, `5b13f2b` , 29,500-character pack	6 match, 2 partial (Q3, Q7), 2 mismatch (Q6, Q10)
Live check at delivery	Ollama on the delivered pack	2026-10-28	Answered in contract and ran as the persona — evidence that the pack fits, not of accuracy

Why 10/10 is weaker evidence than it sounds (L-08)

- The recorded run used the pack **before** the semantic audit; the delivered pack (33,662 characters) was not re-run with Claude at delivery — no Claude CLI on the laptop. Disclosed.
- The pack names the questions ("Relevant to: Q1", "Answers Q2, Q3, Q4, Q7 and Q10 with no further joins"): it tells the model where to look, without numbers. There is no hold-out set.
- One sample per question, one key measure per question, order checked for three.
- The same build session wrote the semantic layer and the reference SQL.

The honest summary: *the semantic layer is good enough for a strong model to answer these ten questions correctly; it does not yet show that it generalises.* The fix is a hold-out set of paraphrased and new questions, question ids removed from the pack, repeated runs and full result-set comparison. (After delivery: four new questions — gender of loans and calendar quarters — were added as a separate suite and answered 4/4 with the verifier active.)

14 Step 12 — Demo, deliverables and the evidence chain

How it is shown

- **Console demo** (`make demo-login`): a seven-act walkthrough — password only → 0 rows; password + code → the user's slice; switching between users; a customer and their manager asking the same question and getting different, correct answers.
- **Web walkthrough** (`make web-open` , `http://localhost:8080`): seven steps — bronze, silver clean, silver grade, gold, access, login, ask. `/api/ask` accepts a completed two-step login only, never a role name. With Ollama it runs fully offline on the laptop.
- **Worked example** (`make demo` , `docs/WORKED_EXAMPLE.md`): one client followed from the line of text she arrives as, through cleaning, into the product and through the entitlement model.

The deliverables and where their numbers come from

Deliverable	Content	Built by
`Berka_TakeHome_Submission.xlsx` (11 tabs)	Overview, Defect types, Model, Questions, Q detail, DQ rules, Cleaning, Assumptions, Decisions, Entitlements, Run log	`make workbook` from the result files and the database
`DDR.pdf` / `DDR.docx` (2 pages)	Seven decisions with options, choice, why, cost; what breaks at 100,000×	`make ddr` ; **fails** if over two pages
`semantic_solution.zip` (183 members)	The semantic layer, code, SQL, tests, results, logs; `MANIFEST.md` says what to read first	`make zip`
`defects.csv` , `model_explanation.md`	The register and the model narrative	`make dq` , `make model`
`docs/DELIVERY_AUDIT.md`	Every claim → its authoritative source → the deliverables that repeat it; commands run; open items	Written at delivery

Every command appends to `dq/results/run_log.jsonl` with its git commit; `STATUS_REPORT.md` is generated from it, and a step that never ran shows `NOT EXECUTED` .

If they ask "how do I know these numbers are real?"

"Every figure in the workbook was produced by a command whose run is logged with its commit. The delivery audit maps each claim to the one place that is authoritative for it. And a step that did not run is printed as NOT EXECUTED, never filled in."

15 The ten questions, one by one

For each question: the delivered reference answer, the interpretation it implements, the trap, how sensitive the answer is to the interpretation, what Claude wrote, and how to say it in two sentences. Reference figures are from the delivered `reference_results.json`; sensitivities were measured on the same data.

Q1 — How many clients do we have per region?

region	clients	pct_of_clients
south Moravia	937	17.45
north Moravia	920	17.14
central Bohemia	664	12.37
Prague	663	12.35
east Bohemia	660	12.29
north Bohemia	561	10.45
west Bohemia	515	9.59
south Bohemia	449	8.36

- **Interpretation:** the region of the **client's own district** (A1), counting **all 5,369 clients**, including the 869 who only hold a disponent relationship (A2). Region via `ai.client` → district.
- **Trap:** counting through dispositions or accounts (4,500, or duplicates); using the account's district; forgetting disponent-only clients.
- **Sensitivity:** places 3–5 are within four clients (664 / 663 / 660), so the order depends on the definition. Owners only, by client district: east Bohemia (556) passes Prague (547). Owners only, by account district: north Moravia (793) passes south Moravia (778). 409 owners live in a different district from their account's (info rule, caveat C4).
- Claude Sonnet 5.5 (recorded run): match 8/8 values, attempt 1 · Ollama qwen2.5-coder:3b (local run): match 8/8 values

```
SELECT
  region,
  COUNT(*) AS client_count
FROM ai.client
GROUP BY
  region
ORDER BY
  client_count DESC,
  region
```

Say it like this

"A client is a person and a person lives in one district, so I count all 5,369 by where they live. 409 owners bank elsewhere; the other reading is one join away and changes places 3 to 5."

Q2 — What is the total amount of loans granted per year?

year	loans	total_amount	average_amount
1,993	20	2,619,276	130,963.80
1,994	101	13,379,904	132,474.30
1,995	90	13,344,372	148,270.80
1,996	117	18,317,676	156,561.33
1,997	196	30,731,364	156,792.67
1,998	158	24,869,148	157,399.67

- **Interpretation:** "granted" is dated by the loan start date, the only date the loan table has; "total amount" is the sum of the **principal**, not payments × duration (A3).
- **Trap:** summing `monthly_payment × duration` (the total repayable, 10–20% higher); dating by the account; treating the date as text.
- **Watch:** 1993 is a half year — the earliest loan is 1993-07-05. 1997 is the peak at 30.7 million. Total 103,261,740 over 682 loans.
- Claude Sonnet 5.5 (recorded run): match 6/6 values, attempt 1 · Ollama qwen2.5-coder:3b (local run): match 6/6 values

```
SELECT
  l.loan_year,
  COUNT(*) AS loan_count,
  SUM(l.amount) AS total_loan_amount
FROM ai.loan AS l
GROUP BY
  l.loan_year
ORDER BY
  l.loan_year
```

Q3 — What percentage of our loans are problematic?

total_loans	problematic_loans	problematic_pct	status_b_finished_unpaid	status_d_running_in_debt
682	76	11.14	31	45

- **Interpretation:** status **B** (finished, unpaid) or **D** (running, in debt), divided by **all** loans (A4, **awaiting business confirmation**). Certified as `is_problematic`.
- **Trap:** only B (31/682 = 4.55%), only D, integer division, mixing count and amount.
- **Sensitivity:** by **value** instead of count it is 15.09% — problematic loans are larger. 95% Wilson interval for the count share: 9.0–13.7%.
- Claude Sonnet 5.5 (recorded run): match 3/3 values, attempt 1 · Ollama qwen2.5-coder:3b (local run): partial 1/3 values

```
SELECT
  COUNT(*) AS loan_count,
  COUNT(*) FILTER(WHERE
    is_problematic) AS problematic_loan_count,
  ROUND(CAST(100.0 * COUNT(*) FILTER(WHERE
    is_problematic) / COUNT(*) AS DECIMAL), 2) AS problematic_loan_share_pct
FROM ai.loan
```

Q4 — What is the average loan amount per card type of the account owner?

owner_card_type	loans	average_loan_amount	total_loan_amount
junior	21	221,970.29	4,661,376
gold	16	161,880.75	2,590,092
no card	512	148,864.71	76,218,732
classic	133	148,808.57	19,791,540

- **Interpretation:** the card on the **owner's** disposition (A5); loans whose owner has no card are an explicit `no card` group, so the counts add up to 682 (A6). Per loan, not per card.
- **Critical reading:** *junior* tops it on only 21 loans whose owners average 17.4 years old at loan start (10 under 18). It is mechanically correct and probably a data artefact, not an insight — report it with n and the age caveat. On this data no card is held by a disponent, so "owner's card vs any card" changes nothing.
- Claude Sonnet 5.5 (recorded run): match 4/4 values, attempt 1 · Ollama qwen2.5-coder:3b (local run): match 4/4 values

```
SELECT
  owner_card_type,
  COUNT(*) AS loan_count,
  ROUND(CAST(AVG(amount) AS DECIMAL), 2) AS average_loan_amount
FROM ai.loan
GROUP BY
  owner_card_type
ORDER BY
  owner_card_type
```

Q5 — Which 10 districts had the highest total incoming transaction amount in 1997?

rank	district_id	district_name	region	total_incoming_amount	incoming_transactions	accounts
1	1	Hl.m. Praha	Prague	115,130,200.10	13,611	554
2	70	Karvina	north Moravia	28,506,437.10	3,802	152
3	74	Ostrava - mesto	north Moravia	27,689,271.90	3,378	135
4	54	Brno - mesto	south Moravia	23,095,083.40	3,041	128
5	64	Zlin	south Moravia	19,723,168.50	2,299	92
6	68	Frydek - Mistek	north Moravia	19,087,598.80	2,006	83
7	72	Olomouc	north Moravia	15,916,397	2,129	88
8	5	Kolin	central Bohemia	14,471,603.60	1,542	65
9	52	Usti nad Orlici	east Bohemia	14,085,167.40	1,431	57
10	19	Prachatice	south Bohemia	12,163,147	1,359	55

- **Interpretation:** credits only (`direction = 'credit'`, source `PRIJEM`), unsigned amount, postings dated 1997, grouped by the **account's** district (A7, A8). Ranked with `rank()` so ties would show.
- **Trap:** treating `VYBER` as incoming; summing `signed_amount` (net flow is a different question); using the client's district without saying so.
- **Sensitivity:** excluding bank interest (`UROK`) keeps the same ten in the same order (A7; whether interest is "incoming money" is an open business question). By the owner's district, ranks 2–3 swap and the tenth place changes. Number 11 (Louny, 12,134,490.00) is 28,657 behind.
- Claude Sonnet 5.5 (recorded run): match 10/10 values, attempt 1 · Ollama qwen2.5-coder:3b (local run): match 10/10 values

```
SELECT
  t.account_district_id,
  t.account_district_name,
  t.account_region,
  SUM(t.amount) AS incoming_amount_czk,
  COUNT(*) AS credit_postings,
  COUNT(DISTINCT t.account_id) AS accounts_with_credits
FROM ai.transaction AS t
WHERE
  t.direction = 'credit'
  AND t.trans_date ⩾ CAST('1997-01-01' AS DATE)
  AND t.trans_date < CAST('1998-01-01' AS DATE)
GROUP BY
  t.account_district_id,
  t.account_district_name,
  t.account_region
ORDER BY
  SUM(t.amount) DESC,
  t.account_district_name ASC
LIMIT 10
```

Q6 — How many new accounts were opened per month in 1996?

year_month	month_number	month_name	accounts_opened
1996-01	1	January	106
1996-02	2	February	104
1996-03	3	March	124
1996-04	4	April	101
1996-05	5	May	91
1996-06	6	June	113
1996-07	7	July	137
1996-08	8	August	133
1996-09	9	September	108
1996-10	10	October	115
1996-11	11	November	106
1996-12	12	December	125

- **Interpretation:** accounts by month of `open_date`, all twelve months shown, driven from the calendar so a month without accounts would read 0 (A9). Total 1,363.
- **Trap:** grouping only accounts (an empty month disappears); grouping month names across years; the fan-out the local model fell into (joining to every day of the month).
- Claude Sonnet 5.5 (recorded run): match 12/12 values, attempt 1 · Ollama qwen2.5-coder:3b (local run): mismatch 0/12 values

```
SELECT
  c.year_month AS month,
  c.month_start,
  COUNT(a.account_id) AS new_accounts
FROM (
  SELECT DISTINCT
    year_month,
    month_start
  FROM ai.calendar
  WHERE
    year = 1996
) AS c
LEFT JOIN ai.account AS a
  ON a.open_date ⩾ c.month_start
  AND a.open_date < (
    c.month_start + INTERVAL '1 MONTH'
  )
GROUP BY
  c.year_month,
  c.month_start
ORDER BY
  c.month_start
```

Q7 — Average age of the account owner at loan start, per loan status

status_code	loan_status	loans	owners_with_known_age	average_age_at_loan_start	average_age_excluding_flagged	loans_flagged_implausible_age
A	finished, paid in full	203	203	37.07	37.53	4
B	finished, unpaid (default)	31	31	39.60	39.60	0
C	running, no problem	403	403	38.70	38.82	2
D	running, client in debt	45	45	37	37	0

- **Interpretation:** the owner (OWNER disposition, A11) at the loan date, in **exact fractional years** (A10); the six loans to owners recorded as under 15 are reported with and without (warn rule, caveat C3).
- **Trap:** not decoding the women's birth month (+50); using today's date; truncating to whole years (about 0.5 years low); using a disponent.

- **Small difference to know:** Claude's answer shows B = 39.61 against the reference 39.60, because `ai.loan` rounds each row's age to two decimals before the model averages it. Within the 0.011 tolerance; name it if asked (L-13).
- Claude Sonnet 5.5 (recorded run): match 4/4 values, attempt 1 · Ollama qwen2.5-coder:3b (local run): partial 1/4 values

```
SELECT
  loan_status,
  status_code,
  COUNT(*) AS loan_count,
  ROUND(CAST(AVG(owner_age_at_loan_start) AS DECIMAL), 2) AS average_age_at_loan_start,
  COUNT(*) FILTER(WHERE
    'val_owner_age_at_loan_start' = ANY(
      data_quality_flags
    )) AS flagged_under_15_loans,
  ROUND(
    CAST(AVG(owner_age_at_loan_start) FILTER(WHERE
      NOT (
        'val_owner_age_at_loan_start' = ANY(
          data_quality_flags
        )
      )) AS DECIMAL),
    2
  ) AS average_age_excluding_flagged
FROM ai.loan
GROUP BY
  loan_status,
  status_code
ORDER BY
  status_code
```

Q8 — Gender split of account owners holding a gold card

gender	gold_card_owners	distinct_clients	pct_of_gold_card_owners
male	53	53	60.23
female	35	35	39.77

- **Interpretation:** owner rows whose own disposition holds a gold card (A12); the unit is the owner row, with distinct clients reported beside it (A13). Gender is derived from the birth number (A18, D016) — labelled derived, not observed.
- **Robustness:** all 88 gold cards are on owner dispositions; no client holds two.
- Claude Sonnet 5.5 (recorded run): match 2/2 values, attempt 1 · Ollama qwen2.5-coder:3b (local run): match 2/2 values

```
SELECT
  owner_gender AS gender,
  COUNT(*) AS owner_count,
  ROUND(CAST(100.0 * COUNT(*) / SUM(COUNT(*)) OVER () AS DECIMAL), 2) AS share_pct,
  SUM(COUNT(*)) OVER () AS total_gold_card_owners
FROM ai.account_owner
WHERE
  owner_card_type = 'gold'
GROUP BY
  owner_gender
ORDER BY
  owner_gender
```

Q9 — Average account balance at the end of December 1998

accounts_with_known_balance	average_balance	median_balance	min_balance	max_balance	accounts_carried_forward	accounts_balance_unreconciled	accounts_overdrawn
4,500	43,808.94	38,494.55	-25,821.20	138,317.20	76	55	39

- **Interpretation:** each account's balance after its last posting on or before 1998-12-31, from the month-end fact (A14); the average is over accounts with a known balance — none counted as zero (A15). 76 accounts had no December posting and carry November forward.
- **Typical wrong answers:** joining on `max(date)` without a tie-break duplicates rows (4,707 rows, average 43,508.31); only accounts with a December posting (4,424, 44,209.53); averaging every December posting (23,553 rows, 46,908.00).
- **The known limitation:** same-day ties are broken by the highest `trans_id`, which is not chronological (C2). Disclosed at delivery. Under the balance-chain rule implemented after delivery the average is 43,810.47 (+1.53 CZK); 107 accounts close differently, by up to 1,200.00 CZK — the "missing posting" of caveat C1 (L-01, L-02).
- Claude Sonnet 5.5 (recorded run): match 1/1 values, attempt 1 · Ollama qwen2.5-coder:3b (local run): match 1/1 values

```
SELECT
  CAST('1998-12-31' AS DATE) AS month_end_date,
  ROUND(CAST(AVG(closing_balance) AS DECIMAL), 2) AS average_month_end_balance,
  COUNT(*) AS accounts_with_balance,
  COUNT(*) FILTER(WHERE
    is_carried_forward) AS carried_forward_balances,
  COUNT(*) FILTER(WHERE
    'con_balance_reconciliation_material' = ANY(
      account_data_quality_flags
    )) AS accounts_with_reconciliation_flag
FROM ai.account_month_end_balance
WHERE
  month_end_date = CAST('1998-12-31' AS DATE)
```

Say it like this

"43,809 CZK over all 4,500 accounts, 76 of them carried forward from November. I disclosed that the closing posting of a busy day is decided by a tie-break; after submitting I replaced it with the running-balance chain — the average moves by 1.53 CZK, individual accounts by up to 1,200."

Q10 — Which statement frequency has the highest share of problematic loans?

statement_frequency	loans	problematic_loans	problematic_pct	rank_by_problematic_share
after transaction	32	6	18.75	1
weekly	91	11	12.09	2
monthly	559	59	10.55	3

- **Interpretation:** a **within-frequency rate** — problematic loans divided by all loans of accounts with that frequency (A16) — not each frequency's share of all problematic loans.
- **Trap:** the share of all problematic loans makes *monthly* the winner (59 of 76 = 77.6%): the wrong question. The local model made a related mistake (absolute count → monthly).
- **Critical reading:** the winner rests on 6 of 32 loans; 95% Wilson interval 8.9–35.3%; Fisher exact test against the rest, two-sided p ≈ 0.155. *Highest observed, not significant.*
- Claude Sonnet 5.5 (recorded run): match 3/3 values, attempt 1 · Ollama qwen2.5-coder:3b (local run): mismatch 0/3 values

```
SELECT
  statement_frequency,
  COUNT(*) AS loan_count,
  COUNT(*) FILTER(WHERE
    is_problematic) AS problematic_loan_count,
  ROUND(CAST(100.0 * COUNT(*) FILTER(WHERE
    is_problematic) / COUNT(*) AS DECIMAL), 2) AS problematic_loan_share_pct
FROM ai.loan
GROUP BY
  statement_frequency
ORDER BY
  problematic_loan_share_pct DESC,
  statement_frequency
```

Say it like this

"After-transaction statements have the highest observed rate, 18.75% — but on 32 loans with an interval of 9 to 35%, so I would not tell the business it is riskier, only that it is worth watching."

16 What breaks at 100,000× the volume

The DDR's answer, reasoned rather than benchmarked: about 100 billion postings, 450 million accounts and some 10 TB of facts before indexes.

#	What breaks	Why	What it becomes
1	**Ingestion and rebuilds**	`core` is rebuilt with `CREATE TABLE AS` on every run: hours rewriting unchanged data	Incremental: facts range-partitioned by month; only open partitions rebuilt; merge, not replace
2	**The data quality runner**	124 full scans; violating keys sampled through Python	Partition-scoped rules (check what arrived); keys written by `INSERT … SELECT`; per-account state for the balance reconciliation; `info` rules sampled with a stated confidence; `block` stays exact
3	**The month-end fact**	450 m accounts × 72 month ends = 32 bn rows rebuilt from scratch	A snapshot maintained from the previous month plus that month's postings (and the chain ordering decided once, at ingest)
4	**Latency and concurrency**	Ad-hoc generated SQL over 10 TB is a full scan unless it prunes; many users exhaust one node and a role-per-user pool	Partitions and indexes on certified time columns; pre-aggregated marts for certified metrics; per-tier timeouts; a pooled gateway doing `SET ROLE`
5	**The entitlement predicate**	A hash join over 4,500 accounts today; over millions for a large portfolio	A materialised `(role, account_id)` set, refreshed on change and co-partitioned with the facts. The design does not change: still a policy; onboarding still an insert
6	**Evaluation and operations**	Reference queries scanning everything are too slow per build; ten questions are not a test set	Frozen samples with checksums; a hold-out question set; monitoring of guard refusals, query cost, rule trends and entitlement changes

What does not break: where the filter sits, and the semantic layer's size — about 8,400 tokens at any volume (though many more entities would need retrieval instead of one pack). A single PostgreSQL node is the wrong host for 10 TB: PostgreSQL with Citus, or a columnar MPP engine (ClickHouse, StarRocks), carrying the same schemas, rules and policy model, is the port to evaluate — or a cloud warehouse's row access policies, with the semantic layer and the policies moving as contracts.

Say it like this

"At a hundred thousand times, everything that is 'rebuild from scratch' becomes 'maintain incrementally' — the core facts, the rule runs, the month-end snapshot — and the entitlement join becomes a materialised set. What does not change is the design: the filter stays in the engine, onboarding stays an insert, and the semantic layer stays the same size."

17 Question bank

The questions most likely to come up in the 15 minutes, with a short model answer. Rehearse them out loud; the goal is the first two sentences, not the whole paragraph. IDs in brackets point to decisions (D), assumptions (A) and lessons (L).

Brief, scope and design

1 Walk me through the architecture in two minutes.

Six schemas, each with a contract: raw (verbatim, hashed) → stg (typed, decoded, nothing filtered) → dq (rules, quarantine, flags) → core (star schema = stg − quarantine, verified each build) → ai (business-named security-invoker views) — with sec beside core holding who may see what, read by seven RLS policies. Then the semantic layer, the guard and the login on the AI path.

2 The brief said 1–2 days and warned against infrastructure. Why 2FA, a web app, a deployment guide?

The three tasks were built and evaluated first. The extras were added on request, and each demonstrates a governance point — the second factor enforced where the data is, onboarding as a data change — while the stack stayed minimal: one database, SQL files, plain Python. What I would cut first: the deployment guide and the simulated agents, then the web page. (L-19)

3 Why PostgreSQL rather than DuckDB or a warehouse?

Task 3 is enforcement. The honest proof that generated SQL cannot bypass a filter is a filter in the engine and a test that connects as a restricted user. DuckDB has no row level security. At 100,000× I would change the engine, not the pattern. (D002)

4 What does "AI-ready" mean in your solution?

Clean typed data with one definition per term; business names; caveats that reach the answer; a semantic layer the build validates; and an entitlement filter the model cannot see or change. Not "we added an LLM".

5 Which single assumption, if wrong, invalidates the most?

The owner definition: one OWNER disposition per account. It underlies Q4, Q7, Q8 and the ownership grant. It is checked by a block rule (4,500 owners for 4,500 accounts). A close second is the sign convention (PRIJEM credit; VYDAJ and VYBER debit), confirmed by the running balance. (A5, A11, A29)

6 What would you do with one more day?

Fix the month-end tie-break with the balance chain and a test; add intent verification so a safe-but-wrong query is refused; fix the disponent card visibility and the "no card" coalesce; run a hold-out evaluation. I have since done the first two on a branch. (L-01, L-07)

Data quality and cleaning

7 Give one block, one warn and one info, and why.

Block: a duplicate posting by business key — it double-counts a flow, so it is quarantined. Warn: owner under 15 at loan start (6 loans) — impossible, but removing the loans would break loan totals used by four other questions, so flag it. Info: 14 zero-amount postings — real interest runs that accrued nothing; changes no sum.

8 Your only blocked row is a 0.00 CZK posting. What did blocking protect?

Nothing measurable in money — and the rule's construction explains why: it requires the same resulting balance, so it can only catch duplicates that did not move the balance. A real duplicated payment would change the balance and slip past it. A warn rule on repeated identical batches is the fix; 1,081 groups of repeated identical interest postings would need a business decision. (L-04)

9 Why never repair a value?

Because a repaired value is indistinguishable from an observed one a week later, and someone will eventually reconcile a number against a statement. The original stays in a _raw column with a note; the rules decide what reaches the product. (D013)

10 How do you know the cleaning lost nothing?

raw = stg on eight row counts and three money sums; core = stg − quarantine on seven row counts and two money sums, after every build. The one quarantined row shows in the row count, not in money (it is 0.00).

11 124 rules — how many are meaningful?

89 are written by hand with a rationale; 35 are a generated completeness sweep. 19 fire on the public copy; the other rules are proven by 20 seeded defects. I would not defend 124 as a quality metric — the rationale per rule is the point.

12 What does "graded, not gated" mean?

Staging never filters. If it dropped the duplicate, the uniqueness rule would have nothing to count and the report would say the data is clean. Grading after the fact makes the report describe the source. (D020)

13 What is the intra-day ordering problem?

The source has no time of day and trans_id is not chronological, so the closing posting of a busy day is ambiguous. I broke ties by the highest id and disclosed it. Later I found the running balance does record the order — each posting starts where the previous one ended — and the chain resolves 98.8% of cases. The honest part: I accepted "unknowable" without testing the chain. (L-01)

14 What was the most useful bug?

The rule self-test nearly deleted real data: the quarantine view was not scoped by schema, so injected defects excluded real rows — 76 districts instead of 77. The model checks failed before any deliverable was written. Verify a model; do not trust a green build. (D008)

Model and SQL

15 Why a star schema with a bridge?

The questions live at four grains; a disposition is many-to-many with a type. A wide table would turn every count into count(distinct), which generated SQL gets wrong quietly. Data Vault is for many sources with history; there is one source and one snapshot.

16 Explain v_account_owner.

The one place that decides "the owner": the OWNER disposition, one per account. Q4, Q7, Q8 and the pre-joined ai.loan all go through it, so the definition cannot fork. Under RLS, for a disponent, the inner join to the filtered client dimension drops the owner — which is where the "no card" problem comes from. (L-06)

17 How is the month-end balance built, and is it right?

One row per account per month end from the first posting month: the balance after the last posting in the month, carried forward with a flag when the month is empty. The average is robust; individual balances on busy closing days were not, because of the highest-id tie-break. (D019, L-01)

18 Why natural keys?

One source, one extract, no history. Surrogates buy conformance and SCD tracking, which this data does not need; they would add a join per dimension and longer SQL for the model. With history: surrogates plus SCD2 on client, account and district. (D007)

19 Why is security_invoker on every view?

Otherwise a view runs as its owner, who is exempt from the policies. That leaked once: ai.loan returned all 682 loans to a user entitled to none. A test now reads pg_class.reloptions and fails if any view between consumer and policy lacks it. (D010)

The ten questions

20 Q1: which district, and what changes with the other one?

The client's own district, all 5,369 clients. Places 3–5 are within four clients; owners only by account district, north Moravia passes south Moravia. Which one the business means depends on the use: marketing (where people live) or branch performance (where accounts are). Ask. (A1)

21 Q4: junior card owners borrow the most. Do you believe it?

It is mechanically correct — 21 loans averaging 221,970 — but those owners average 17.4 years old at loan start. I would report it with n and the age caveat as a data artefact, not a finding.

22 Q5: does incoming include interest?

Yes — PRIJEM includes the bank's interest credits. Excluding them keeps the same top ten in the same order; whether interest counts as incoming money is still a business decision. (A7)

23 Q9: what is account 31's balance at the end of November 1998?

The delivered fact says 13,312.90 — the balance after interest. The 14.60 statement fee came after it in the chain, so the true closing balance is 13,298.30. That is the tie-break limitation, and it carries forward into December. (L-01)

24 Q10: is "after transaction" really riskier?

Highest observed, not significant: 6 of 32 loans, interval 8.9–35.3%, Fisher p ≈ 0.16. I would say it is worth watching, not that it is riskier.

25 Which answer are you least confident in?

Q10 (small n) and Q4 (an odd population) for interpretation; Q9 at account level, because of the tie-break.

Semantic layer, LLM and evaluation

26 How do you know the LLM isn't just reading your answers?

No reference figure is in the pack — a test asserts it — and the model had no database, no files and no tools. But the pack does tell it which entity answers which question; that shows it where to look, not what the answer is. A hold-out set is the remedy. (L-08)

27 Is 10/10 meaningful?

It is a narrow pass: one sample per question, one key measure compared, questions known to the author of the layer, run on the earlier pack. It shows the layer is good enough for a strong model on these questions; it does not show generalisation. The local 3B model's 6/10 shows what weaker reasoning does with the same layer.

28 Does passing the guard mean the query is right?

No. The guard proves a query is safe — one read-only SELECT on the governed surface, bounded in rows and time. It does not resolve columns or judge meaning. The gender question proved it: guard allowed, database rejected an unknown column. (L-07)

29 Why sqlglot rather than a regex?

Comments, casing and string literals defeat string matching. SELECT/**/1; DROP TABLE x is one line to a regex and two statements to a parser.

30 What does the semantic validator catch?

A described column that does not exist, an undescribed one, a false grain, a wrong join cardinality, a stale row count, and a metric naming a missing column. It does not check whether the model used the layer correctly — that is what intent verification adds. (D029)

31 What happens when a disponent asks Q4?

Their one loan shows owner_card_type = 'no card' and a NULL frequency, because the owner is hidden by RLS and the view coalesces. The answer is wrong, not just restricted. The fix: no coalesce on hidden joins, or an explicit "not visible". (L-06)

32 Why run the LLM as cro_global?

A portfolio of every client, so the answers are comparable with the reference — still through the policies (rolbypassrls = false). The same SQL as rm_north returns her slice.

33 Isn't 0.011 tolerance loose?

It absorbs two-decimal rounding and nothing larger. A relative 1e-4 — the first version — would have accepted the top Q5 district wrong by 11,513 CZK. It did absorb one real difference I can name: Q7 status B, 39.61 vs 39.60, from rounding before averaging.

Entitlements and authentication

34 Where does the filter sit, and why can't the LLM bypass it?

Policies on the seven client-bearing core tables, evaluated as the caller through security-invoker views, keyed on session_user. The model emits text; the predicate is not in the text; no consumer has bypass or superuser. Nine bypass tests fail as they should.

35 Onboard a new relationship manager without code.

Insert into sec.app_user and sec.user_portfolio, create the login role with its credential, enrol the TOTP factor. Grants are derived live, so access works immediately. Offboarding is one UPDATE. (D027)

36 Can a disponent see anything about the owner?

The owner's client record, no. But the owner's card (type, issue date, holder id) and the owner's disposition row, yes — the card and disposition policies key on the account. The documents over-promise here; the fix is a holder-based policy on cards and own-row-only dispositions, with tests. (L-05)

37 How is the second factor enforced, and what are its limits?

has_verified_session() is ANDed into the entitlement resolution; the verify function takes only a code and uses session_user; codes are single use; five committed failures lock for 15 minutes. Limits: a direct client can roll back failed guesses (found and disclosed at delivery), the session is bound to the backend PID only, secrets are stored recoverable, TOTP is phishable. (L-09, L-10)

38 A role per user does not pool. What then?

A pool per role, or a trusted gateway that does SET ROLE on a privileged connection — the gateway, not the model, chooses the role. Identity then moves from session_user to current_user and the session binding from PID to a gateway token: a design change to plan and test.

39 What is missing for a real bank?

A read audit log, column masking, secrets encrypted with an external key, joiner-mover-leaver automation, periodic access recertification from v_access_grant, and a break-glass procedure.

40 Is the public web demo secure?

No, by design: it shows the one-time codes, personas share a password, there is no rate limit, and the process holds owner credentials for the tour pages. Acceptable only because Berka is public 1999 data, and labelled as a demo affordance.

Testing and operations

41 How many tests, and what do they prove?

At delivery the full suite passed with 327 passed and 1 skipped (the opt-in live-model test); the auth module 40 passed and 1 strict expected failure that records the rollback gap. Collected count 330. They prove rules fire, the model's grains and reconciliation hold, the guard refuses hostile SQL, RLS cannot be bypassed and the second factor is required. Older documents quote 137, 202 or 241. (L-14)

42 What is not tested?

Disponent card and disposition visibility, RLS-induced NULL semantics, PID reuse, the who-can-reach NULL branch, repeated interest batches — and, in the delivered version, the tie-break against the balance chain and the meaning of generated SQL.

43 How do I reproduce it?

make up && make all on Linux or Docker; scripts\windows.ps1 build on native Windows with a local PostgreSQL; make build runs everything except the LLM steps. The run log records every command with its commit.

44 Why did you not merge your later fixes into the delivery?

Because they change a certified number (Q9) and the reference results, and a delivery freeze is the wrong moment to change a certified number without the business signing off the new rule. So the delivery discloses the limitation, and the fix sits on a branch with its tests and before/after evidence. (Lessons Learned, chapter 8)

When you do not know

1. Say what you do know, with the evidence: "I measured X; I did not measure Y."
2. Say how you would find out: a concrete query or test.
3. Do not guess numbers. "I don't have that figure; the closest I have is ..."
4. If it exposes a flaw, own it and give the fix and its cost.

18 Decision register

Every decision of the delivered version, D001–D027, generated from the delivered `DECISIONS.md`: context, options, choice, reason, cost and consequence. D022 is superseded by D023 and kept on purpose. The two decisions taken after delivery follow at the end, marked as such.

Decisions D001–D012: platform, data, model, security foundations

D001 · Repository lives at the git root, not in a nested project folder

CONTEXT The brief asks for "a Git repository or zip file following the structure in section 4". The working repository is `Loan-valid-DQ`.

OPTIONS (a) Nest everything under `berka-governed-data-product/`. (b) Put the project at the repository root.

CHOICE (b) Project at the root.

REASON The repository *is* the submission. A nested folder adds one path segment to every command in the README and buys nothing; `make all` from the clone root is what a reviewer will try first.

CONSEQUENCE `src/config.py` derives `PROJECT_ROOT` from its own location, so the layout can still be moved without touching any other module.

D002 · PostgreSQL 16, run natively where no Docker daemon exists

CONTEXT Task 3 is about entitlement-aware access. The only way to show that an LLM-generated query *cannot* bypass the filter is to put the filter in the database engine and prove it with a real connection as a real low-privilege user. The build container has the Docker CLI but no Docker daemon (`/var/run/docker.sock` is absent), and it does have a PostgreSQL 16 cluster.

OPTIONS (a) PostgreSQL 16 in `docker compose`. (b) DuckDB file, shipping the PostgreSQL row level security DDL as untested scripts. (c) PostgreSQL 16 from the native cluster, keeping `docker-compose.yml` for portability.

CHOICE (c), with (a) as the preferred path when a daemon is present. `scripts/db_up.sh` tries `docker compose` first and falls back automatically.

REASON DuckDB has no row level security, so option (b) would reduce Task 3 to prose and untested SQL -- the weakest possible answer to the one task that is purely about enforcement. The native cluster runs the identical DDL, identical policies and identical tests, so nothing about the submission is hypothetical.

COST One shell script (about 50 lines) to paper over the difference, and a `make up` that is slightly less hermetic than a pure compose stack.

CONSEQUENCE `make up` works in both worlds. Every row level security claim in this submission is backed by a test that actually connects as a restricted role (`tests/test_rls.py`).

D003 · Data acquired from a verified public GitHub mirror of the PKDD'99 files

CONTEXT The documented acquisition order was: the CTU relational repository (MySQL on `relational.fel.cvut.cz:3306`), then the author's archive at `sorry.vse.cz`, then a Kaggle mirror. In this environment outbound traffic goes through a policy-enforcing proxy that allows HTTPS to a host allow-list only: `sorry.vse.cz` and `relational.fel.cvut.cz` are both refused (HTTP 403 on CONNECT), and raw MySQL on port 3306 cannot be tunnelled at all.

OPTIONS (a) Stop and ask for the files to be placed in `data/raw/`. (b) Use a public mirror that is reachable, and verify it is byte-equivalent in substance to the original.

CHOICE (b) `github.com/jlacko/berka-dataset`, which ships the original eight `*.asc` files (semicolon separated, quoted headers) plus the original "Financial Data description.pdf".

REASON The mirror is verifiable against published figures, which is what matters for trust: row counts came out at 4,500 accounts / 5,369 clients / 5,369 dispositions / 892 cards / 682 loans / 77 districts / 6,471 orders / 1,056,320 transactions -- every one an exact match to the reference counts in the brief's dataset. The files are the unmodified `.asc` originals, not a re-exported CSV with renamed columns, so no column mapping guesswork is needed.

CONSEQUENCE `raw.load_audit` stores a SHA-256 per source file, so the exact bytes behind every number in this submission are identifiable. `trans.asc` is stored gzipped (69 MB → 17 MB) and hashed on its *decompressed* stream, so the hash identifies the data rather than the compression settings.

D004 · Public copy is primary; defects are injected reproducibly in a separate schema (HD1)

CONTEXT The brief says the bank "may have injected extra defects into its copy", and the assessment compares findings against those injected defects. No bank-supplied copy was received, so `data/raw/provided/` is empty and the public copy matches the reference counts exactly -- meaning it carries no injected defects beyond the ones the original dataset has always had.

OPTIONS (a) Run the rules on the clean public copy only, accepting that most rules report zero and the defect tab is thin. (b) Also build a reproducible defect injector in a separate schema and report both.

CHOICE (b) -- confirmed with the candidate at decision point HD1.

REASON A rule that has never fired is not evidence of anything. The injector (`sql/70_inject_defects.sql`, seeded and deterministic) corrupts a copy of `stg` in schema `inject` with one defect per rule family, and `tests/test_dq.py` asserts that the matching rule catches each one. The certified answers to the 10 questions are still computed on the untouched data, and every injected row is labelled as injected everywhere it appears.

CONSEQUENCE Two numbers per rule in the report: what fired on the real data, and what fired on the injected copy. When the bank's own copy arrives, `make dq` on `data/raw/provided/` is the only step needed -- the rules do not change.

D005 · The LLM is the local Claude CLI, called as a sandboxed text completion (HD2)

CONTEXT Task 2 requires an LLM to generate the SQL for the 10 questions using the semantic layer. The documented default was a local open-weight model through Ollama; Ollama is not installed in the build container and the model download hosts are not on the proxy allow-list.

OPTIONS (a) Hosted API behind `LLM_PROVIDER` / `LLM_MODEL` / `LLM_API_KEY`, which needs a key this environment does not hold. (b) Export the prompt pack and mark the step `NOT EXECUTED`. (c) Use the authenticated Claude CLI that is present, invoked as a plain completion endpoint.

CHOICE (c) -- confirmed with the candidate at decision point HD2. `src/llm_client.py` keeps all three behind one interface, so switching provider is one environment variable.

REASON The point of the exercise is whether the *semantic layer* is good enough for a model to produce correct SQL. That needs a real model call. The CLI is invoked with `--strict-mcp-config`, an empty MCP config, an explicit system prompt and every tool disallowed, so the model has no database connection, no filesystem and no search: it sees only the rendered semantic context pack and the question, exactly like an API call would. Without that sandboxing the result would prove nothing, because an agent could simply go and read the tables.

COST One subprocess wrapper, and a dependency on a CLI rather than an SDK.

CONSEQUENCE Every call writes `llm_logs/<question>.json` with the full prompt, model id, parameters, raw response and timing. Nothing in the Questions tab is hand-written: where a call fails, the cell says `NOT EXECUTED`.

D006 · Code-to-label mapping lives in one place, and the redundancy is checked

CONTEXT The seven Czech code sets (statement frequency, loan status, card type, disposition type, transaction type, operation, category) have to be readable in staging for profiling, available in SQL for the model, and available to the LLM in the semantic layer. Three places to write the same mapping.

OPTIONS (a) Repeat the mapping in each place. (b) One source, and derive the rest. (c) One source for the product, with any convenience copy verified.

CHOICE (c). `src/config.py` holds the mapping; `src/build_model.py` publishes it as `core.ref_codes`; `sql/30_core.sql` joins that table for every label; the semantic layer renders its allowed-value lists from the same place. The staging layer additionally decodes labels inline while casting, which is a convenience copy -- so `build_model.check_codes_against_staging` asserts that staging's labels equal `ref_codes`, and that no code exists in the data that the catalogue does not publish.

REASON The failure mode that matters is not duplication, it is *silent* duplication: two places disagreeing and nobody noticing until a report and a dashboard show different words for the same loan. A checked copy fails the build instead.

COST One verification function, about 60 lines, and one extra join per labelled column in the core build.

D007 · Natural keys, not surrogate keys

CONTEXT The source keys (`account_id`, `client_id`, `trans_id`, ...) are stable integers with no gaps in meaning and no reuse.

OPTIONS (a) Surrogate integer keys per dimension with the natural key as an attribute. (b) Use the natural keys as primary keys.

CHOICE (b).

REASON Surrogate keys buy conformance across multiple source systems and the ability to track slowly changing dimensions. There is one source system here and a single extract with no history, so they would buy a join per dimension and nothing else. They also make every example query longer, which matters when the queries are written by an LLM reading a context pack.

CONSEQUENCE At 100,000x this is one of the first things to revisit -- along with partitioning the transaction fact by month. It is called out in the Design Decision Record for exactly that reason.

D008 · The rule self-test nearly deleted real data; the fix is schema scoping

CONTEXT `core` is built from `stg` minus the rows block rules rejected, read through the view `dq.v_blocked`. The identical rule set also runs against schema `inject` (the corrupted copy that proves the rules fire), and writes its verdicts to the same `dq.quarantine` table.

CHOICE `dq.v_blocked` and `dq.v_row_flags` filter `target_schema = 'stg'`, and the comment in `sql/00_schemas.sql` says why the filter is load-bearing.

REASON AND CONSEQUENCE This is the argument for verifying a model rather than trusting a build: the pipeline ran green, produced plausible row counts, and was quietly wrong. It is also the argument for keeping the test data in a separate schema rather than mutating staging in place. Kept in this record on purpose -- it is a better answer to "how do you know your pipeline is right" than any clean run.

D009 · The entitlement filter sits in the database, and identity comes from `session_user`

CONTEXT Task 3: expose the data product so each user sees only rows for their own clients, say where the filter sits, say why an LLM-generated query cannot bypass it, and onboard a new user or portfolio without code changes.

CHOICE (c), with (b) kept as a secondary control (`src/sql_guard.py`).

REASON (a) is not a control, it is a request: the thing being filtered is also the thing being asked to apply the filter. (b) is real, but it is defeated by one query shape nobody thought of, and the blast radius of a mistake is every user. (c) puts the predicate inside the plan of every statement on that connection. The LLM never holds the connection -- it emits text -- and the predicate is not in the text, so no rewriting of the text can remove it. `tests/test_rls.py` runs nine bypass attempts (switch role, read the base tables, read the raw layer, set a session variable, edit the entitlement tables, disable the policy, drop the policy, write, join outwards) and all nine fail.

CHOICE (ii).

REASON A GUC can be set by anything that can send SQL, so an entitlement resolved from one is exactly as strong as the guard in front of it -- and the guard is a Python function, which is a weaker place to stand than the engine. `session_user` cannot be re-pointed without a password the model never sees. `tests/test_rls.py::test_setting_a_session_variable_does_not_change_visibility` sets `app.end_user` to another user's name and shows the visible row count does not move.

COST Role-per-user conflicts with naive connection pooling: a pool keyed only by DSN cannot be shared across users. The practical answer is a pool per role (banks run hundreds of roles routinely) or `SET ROLE` performed by a trusted gateway on a privileged connection, where the gateway -- not the model -- decides the role. Discussed in the Design Decision Record.

D010 · Every view between the consumer and the policy must be `security_invoker`

CONTEXT The `ai` views were created `WITH (security_invoker = true)` so that the row level security policies on `core` are evaluated as the calling user. The three curated views in `core` (`v_account_owner`, `v_loan_enriched`, `v_client_region`) were not.

CHOICE Every view on the path from a consumer to a policy carries `security_invoker = true`, and `tests/test_rls.py::test_every_view_between_the_consumer_and_the_policy_is_security_invoker` reads `pg_class.reloptions` and fails if any of them does not. The one deliberate exception, `ai.my_entitlement`, is named in the test with its reason.

REASON AND CONSEQUENCE This is the most useful thing in this submission to be asked about. The design was right, the DDL was one setting short, and the symptom was invisible from the owner's connection -- the only way to see it was to connect as a restricted user and count rows. That is why the entitlement claims here are backed by tests that authenticate as real low-privilege roles, rather than by a description of the policies.

D011 · The LLM runs sandboxed, and the evaluation is value-based

CONTEXT Task 2 asks for an LLM to answer the 10 questions using the semantic layer, and for the generated SQL and the result per question.

REASON An agent that can read the database or the repository would answer the questions by looking, and the exercise would measure the agent rather than the semantic layer. The recorded transcripts in `llm_logs/` show `permission_denials: []` and a single turn per question, which is what a pure completion looks like.

REASON Two correct queries name their columns differently -- `clients` versus `client_count` -- and a correct answer may carry extra supporting columns. Comparing SQL text or column names would measure style. What matters is whether the number the business user reads is right.

HONEST NOTE ON ONE EVALUATOR CHANGE On the first run Q6 scored `mismatch`. The model's counts were identical to the reference for all twelve months; it had labelled the months `1996-01` while the evaluator was looking for the integer `1`. That was a defect in the evaluator, not in the answer, so the evaluator now accepts any published representation of a dimension (a month as `1996-01`, `1` or `January`; a district by name or id). To keep that change from quietly becoming "loosen until it passes", `tests/test_evaluate.py` corrupts the recorded answers and asserts the evaluator rejects them: doubled measures, a reversed ranking, truncated rows, an empty result and a missing answer all fail, and `True` is not allowed to match a count of 1.

D012 · Rebuilding core restores the AI surface in the same target

CONTEXT `sql/30_core.sql` rebuilds the core tables with `DROP TABLE ... CASCADE`, which also drops the `ai` views that depend on them. Running `make model` on its own therefore left the consumer surface missing, which surfaced as `relation "ai.transaction" does not exist` in the middle of the question run.

CHOICE The `model` target runs `src.build_security` after `src.build_model`, so the ai views, the policies and the grants are restored by the same command that invalidated them.

REASON A target that leaves the database in a state no other target expects is a trap for the next person -- and `make all` ordering would have hidden it, because `entitlements` runs after `model` there anyway. The failure only appeared when a single step was re-run, which is exactly how someone iterates.

CONSEQUENCE Both `make model` and `make all` leave a complete, queryable surface. The ai views are cheap to rebuild (they are views), so the extra step costs about a second.

Cleaning decisions (D013-D020)

D013 · Cleaning never repairs a value in place

CONTEXT Eight source files, 1,079,680 rows, with placeholders, undecodable codes, unparsable dates and a balance chain that does not always reconcile. The textbook options for a bad value are repair it, drop it, or keep it and say so.

OPTIONS (a) Repair: impute the missing unemployment rate, infer the intended balance, correct the duplicate. (b) Drop every row with any defect. (c) Type, normalise, decode and derive -- but never overwrite an observation -- and let a separately graded rule layer decide the consequence.

CHOICE (c).

REASON A repaired value is indistinguishable from an observed one a week later. That is tolerable in a model you train and throw away, and not tolerable in a bank's data product where someone will eventually reconcile a number against a statement. Dropping is worse: it silently changes every denominator. The chosen shape means that of the cleaning steps in the ledger, exactly one (C14, quarantine) removes a row from the product, it removed **1**, and C21 proves the arithmetic on both rows and money.

CONSEQUENCE The product carries known-bad rows with flags on them, so a consumer has to be willing to read a caveat. The 55 accounts whose balance chain does not reconcile are still in `core`, still answerable, and still marked -- which is the honest result, and also the one that costs the reader more attention than a quietly cleaned number would.

D014 · Placeholders become NULL; the original text survives beside them

CONTEXT The source writes "missing" at least seven ways: empty string, quoted empty string, a single space, `?`, `NA`, `-`, and the word `unknown`. District 69 (Jesenik) carries `?` in both the 1995 unemployment rate and the 1995 crime count.

OPTIONS (a) Leave the text as it is and let every consumer handle it. (b) Map the known placeholder set to NULL and lose the original. (c) Map to NULL and keep the original in a `<column>_raw` audit column, with a note.

CHOICE (c), with the placeholder list in `src/config.NULL_PLACEHOLDERS` and mirrored by `stg.norm_text`.

REASON Option (a) makes every completeness number wrong, because `?` counts as a value. Option (b) makes the cleaning unfalsifiable. Option (c) costs two columns per affected table and leaves the step checkable: C03 measured **1,380** rows where a placeholder became NULL (1,379 standing orders with no category, and the one district), and each of them can still be read in its raw form.

CONSEQUENCE The audit columns are dead weight on tables with no placeholders, which is most of them. A real warehouse would carry this as one side table rather than per-column, and that is the shape to move to at volume.

D015 · Two-digit years are read as 19xx, and an unparsable date is NULL

CONTEXT Every date in the source is `YYMMDD` with no century. `card.issued` additionally carries a clock time (`931107 00:00:00`) that no other date column has.

OPTIONS (a) A sliding window (`≤ 30` means 20xx). (b) Hard-code the 20th century. (c) Refuse to parse and keep the text.

CHOICE (b), with the trailing clock time stripped before parsing, and a failed parse returning NULL rather than raising.

REASON The dataset window is 1993-1998 and `VAL_DATE_OUT_OF_RANGE` fires if anything lands outside it, so the sliding window would add a rule for a case that cannot occur in this extract. The NULL-on-failure part matters more than the century: a cast that throws aborts the load and hides the defect it tripped over, which is precisely the value a quality report needs to see. C05 parsed **1,062,394** dates and C04 found **0** amounts that failed to cast.

CONSEQUENCE This assumption (A17) is the one that breaks first if the dataset is ever extended past 1999. The fix is one function, and the rule that would catch the breakage already exists.

D016 · Gender and birth date are derived from `birth_number`, and labelled as derived

CONTEXT The source has no gender column and no birth date. It has `birth_number`, a Czech national identifier that packs both: `YYMMDD`, with 50 added to the month for women.

OPTIONS (a) Publish `birth_number` and let consumers decode it. (b) Decode it into `birth_date` and `gender` and drop the original. (c) Decode it, keep the original, and publish a validity flag beside both.

CHOICE (c).

REASON Leaving it packed means every age question re-implements the decoding, and an LLM writing that arithmetic from a context pack will get the month offset wrong. Dropping the original removes the only evidence the decoding was right. C07 decoded **5,369** birth dates and C08 derived **5,369** genders, with `birth_number_valid` true for all of them -- so the derivation is complete on this extract, which is worth knowing precisely because it will not be true of the next one.

CONSEQUENCE Gender here is an inference from an identifier format, not a self-reported attribute, and the ledger and A18 both say so. Any analysis that groups by it inherits that caveat. It is also personal data derived from personal data, which is a question a real deployment has to answer before it publishes the column; this submission publishes it because the ten questions need age, and records the concern rather than pretending it does not exist.

D017 · Codes are decoded to labels from one map, and the code is kept beside the label

CONTEXT Five tables carry Czech codes: `POPLATEK MESICNE`, `VYDAJ`, `PREVOD NA UCET`, `SIPO`, loan status `A` / `B` / `C` / `D`. An LLM asked about "withdrawals" cannot be expected to know `VYDAJ`.

OPTIONS (a) Publish codes only. (b) Replace codes with labels. (c) Publish both, from one map, and fail the build if the data holds a code the map does not know.

CHOICE (c). The map lives in `src/config.py`, is published as `core.ref_codes`, and `build_model.check_codes_against_staging` compares the catalogue against every coded column.

REASON Codes only, and the semantic layer has to carry a glossary the model may ignore. Labels only, and the join back to the source is gone. Both, from one place, and the redundancy becomes a test: if the inline label in staging disagrees with the catalogue, the build stops. C09 labelled **2,466,175** coded values against **28** published codes. **Consequence, and the one this decision was actually written for.** The check is only as good as its list of columns, and the first build's list had `stg.trans.k_symbol_code` in it but not `stg."order".k_symbol_code`. The code `LEASING` therefore sat on **341** standing orders with no English label through the entire first submission. Nothing failed: the profiler reported the code in `dq/results/profile.md`, and the only rule that noticed was the null-label completeness sweep, which reads as "this column is often empty" rather than "a cleaning step did not happen". The fix is three lines and the lesson is bigger than they are: a cleaning guarantee that is enforced per column has to be enumerated per column, and an enumeration maintained by hand will miss one. `LEASING` is now in the map, `stg."order"` is in the catalogue check, `VAL_ORDER_KSYMBOL_DOMAIN` reports unknown order categories the way its transaction twin always did, and `tests/test_dq.py` now asserts both that no staged row carries a code without a label and that every coded column appears in the catalogue check -- so the next coded column cannot be added without being covered.

D018 · The sign of a transaction is resolved once, in staging

CONTEXT `trans.amount` is unsigned. Direction lives in a separate Czech code: `PRIJEM` increases the balance, `VYDAJ` and `VYBER` decrease it. Two different words mean "outgoing".

OPTIONS (a) Publish amount and type, and let each query apply the sign. (b) Publish a `direction` label and let each query apply the sign. (c) Publish `signed_amount` computed once, with amount and type kept beside it.

CHOICE (c).

REASON This is the single most valuable cleaning step in the build for AI-readiness, and it is worth being plain about why. Every "net flow", "total out", "largest withdrawal" question turns on a `CASE` over two Czech words. A query that gets it wrong does not fail -- it returns a confident, plausible, wrong number, and the most likely way for that to happen is an LLM that saw `VYDAJ` in a context pack and did not see `VYBER`. Resolving it once in the data removes the failure mode rather than guarding against it. C10 signed **1,056,320** postings.

CONSEQUENCE One derived column on the largest table, and three columns that can in principle disagree. `VAL_TRANS_AMOUNT_NEGATIVE` and the balance reconciliation rules are what keep them honest. This is also the clearest example of the general point: the way to stop a generated query being wrong is to make the right answer the easy one to write, not to inspect the query afterwards.

D019 · A month-end balance is derived, with the tie-break written down

CONTEXT Q9 asks for a closing balance. The source has no month-end balance; it has postings with a running `balance_after`. Worse, `trans_id` is not chronological within a day, so "the last posting of the month" is genuinely ambiguous for **183** accounts (`CON_INTRADAY_ORDER_AMBIGUOUS`).

OPTIONS (a) Make the consumer derive it per query. (b) Derive it and leave months with no activity missing. (c) Derive it, fill the gaps by carrying the last known balance forward, and mark the carried rows.

CHOICE (c): latest `trans_date` in the month, ties broken by highest `trans_id`, gaps carried forward with `is_carried_forward` and `balance_as_of_month_end` exposed.

REASON (a) guarantees two answers to the same question. (b) makes "average closing balance" silently exclude quiet accounts, which biases it upward. (c) costs one pre-computed table and makes the ambiguity visible rather than arbitrary: C20 shows **558** month ends inherited from an earlier posting, each marked as such.

CONSEQUENCE The tie-break is a choice, not a fact, and for those 183 accounts a different tie-break gives a different number. That is stated in the semantic layer as a caveat so it reaches the answer, and it is the reason the rule is `warn` rather than `info`.

D020 · Cleaning is graded, not gated: nothing is filtered before the rules run

CONTEXT The staging layer could reject bad rows as it types them -- it is the natural place to put a filter, and it would make `core` cleaner.

OPTIONS (a) Filter in staging as part of the cast. (b) Filter nowhere, and publish everything. (c) Stage everything, grade it in a separate layer, and let the grade decide what reaches the product.

CHOICE (c). `sql/10_stg.sql` states it as a contract: *nothing is filtered out here*.

REASON A rule layer cannot report on rows it cannot see. If staging drops the duplicate transaction, the uniqueness rule has nothing to count and the defect report says the data is clean. Separating the two means the quality report describes the source rather than describing what survived the loader -- which is the difference between a data quality report and a data quality anecdote.

CONSEQUENCE `stg` is the larger of the two layers and holds rows that will never be published. That is the intended cost, and it is also what makes C21's reconciliation meaningful: `core` = `stg` - quarantine is checkable precisely because `stg` kept everything.

Privacy at client grain (D021-D022)

D021 · A self-service customer is a principal that IS one client, not one that HOLDS a book

CONTEXT The brief words Task 3 as "each user sees only rows for their own clients", which describes a relationship manager with a book. The sharper question underneath it is whether one bank customer can see another's data. Those are two different grains of principal and the model had only the first.

OPTIONS (a) Give each customer a portfolio containing exactly themselves, so nothing in the model changes at all. (b) Add `subject_client_id` to `sec.app_user` and resolve it in `sec.v_my_clients` alongside the portfolio path. (c) Keep identity out of the database and filter per customer in an application layer above it.

CHOICE (b).

REASON (c) is the option this project already rejected for relationship managers (D009) and it is worse here, not better: a self-service surface is the one most likely to face the open internet, so it is the last place the filter should live in application code. (a) is genuinely tempting -- zero schema change -- but it models a customer as a degenerate book, which means 5,369 portfolios of one row each, an administrative surface where a customer's "book" could be edited to contain somebody else, and a `portfolio_client` table whose rows mean two different things. (b) says what is true: this principal is that client. The cost is one column, one `UNION` and one check constraint.

D022 · The customer's view is narrowed to ownership, matching the manager's (superseded by D023)

CONTEXT `sec.v_my_accounts` resolves accounts from entitled clients `WHERE is_owner` (A21). For a relationship manager that is right: their book is defined by who they are accountable for. For a self-service customer it is a real choice, because 869 clients hold a *disponent* relationship -- they may operate an account they do not own.

OPTIONS (a) Ownership only, identical to the staff rule. (b) Ownership plus disponent, so a customer sees every account they can actually operate.

CHOICE (a), for now, and deliberately.

REASON (b) is arguably the better product: in a real bank, an account you are authorised to transact on appears in your app. But it means the two grains of principal resolve accounts by *different* rules, and a reviewer asking "why does a customer see more kinds of account than their manager" deserves a better answer than "it seemed friendlier". Narrower is also the safer default for a privacy boundary: the failure mode of (a) is a customer who cannot see something they should, which is a support ticket; the failure mode of (b), if the disponent join is ever wrong, is a customer seeing an account that is not theirs.

CONSEQUENCE It is one predicate in one view, not a design change, and the rule to change is named here so the change is a decision rather than a patch. Any real deployment should make this call with the business before launch, because the two readings are both defensible and only one matches what the bank tells its customers their app will show.

Identity, access and authentication (D023-D027)

D023 · A customer sees the accounts they may operate, not only those they own

CONTEXT D022 narrowed a self-service customer to accounts they own, consistent with the staff rule (A21/A36), and recorded that the alternative should be settled with the business. It has been: a customer sees what they are authorised to use.

OPTIONS (a) Ownership only, as before. (b) Ownership plus disposition.

CHOICE (b).

REASON It is what the bank tells its customers their app will show, and the narrow reading produces a support ticket every time somebody cannot see an account they transact on daily. The safety argument for (a) was real but it is answered by *where* the right is granted rather than by withholding it -- see D024.

CONSEQUENCE 869 clients hold a disposition on an account they do not own. In the demo, client 3764 reaches account 3112 and its 439 postings; before this change he reached nothing at all.

D024 · A disposition grants the account, never the person

CONTEXT Having decided a disponent sees the account, the question is what else comes with it. The entitlement chain elsewhere runs client → accounts, so the lazy implementation would grant the client and let the accounts follow.

OPTIONS (a) Grant the owning client, and let their accounts resolve. (b) Grant that one account, and nothing else.

CHOICE (b), and it is the most important line in the access model.

REASON (a) would mean that authorising someone to pay a bill from your account lets them read *you*: your other accounts, your loans, your date of birth. That is not what the arrangement means, and nobody signing it would expect it. The right is over the account, so the grant is over the account.

CONSEQUENCE `sec.v_access_grant` carries `subject_kind` so a grant can be either shape, and disposition grants are always `account`. Proven by `test_a_disponent_reaches_the_account_but_not_its_owner`: client 3764 reads account 3112 and its transactions, and gets zero rows for client 3763.

D025 · Three names for a client, with different exposure

CONTEXT `client_id` is a dense sequential integer and was the only name the product had. Row level security already stops one customer reading another's rows, so exposing it is not a breach -- but it is guessable, it leaks the size of the book, and it is a stable handle for correlation.

OPTIONS (a) Keep `client_id` as the only identifier and rely on the policies. (b) Replace it with an opaque reference. (c) Keep the integer internally and add an opaque external reference, plus a recognisable account number.

CHOICE (c): `client_id` for joins, `client_ref` (`BRK-XXXX-XXXX` , HMAC under a secret salt) to quote, `account_ref` (IBAN with real mod-97 check digits, fictional bank code) for the number a customer recognises.

REASON (a) leaves an enumeration surface that costs nothing to remove: every guess is an access attempt the system must refuse rather than one never made. (b) would make every join and every policy predicate longer and slower for no gain, because the internal key is never exposed anyway. (c) separates the identifier used for plumbing from the identifier used for people, and the semantic layer tells the model which is which.

CONSEQUENCE The salt is set only on the owner's session, so a consumer cannot derive another client's reference from an id. A rebuild under a different salt reissues every reference, which is the intended behaviour if one is believed to have leaked. The IBAN check digits are genuine, so a typo in a demo fails the way a typo in production would.

D026 · The second factor is enforced by the database, not by an application

CONTEXT The brief asks for entitlement-aware access; a self-service customer surface raises the next question, which is how the system knows the person at the other end is who the role says they are.

OPTIONS (a) Authenticate in an application gateway and let it open a trusted connection. (b) Record that two-factor authentication "would be required" and demonstrate the entitlements only. (c) Enforce it in the database: no verified session, no rows.

CHOICE (c).

REASON (a) is where most real systems put it, and it is a defensible production architecture -- but it makes the control a property of the application rather than of the data, which is the same mistake this project rejected for entitlements in D009. (b) is a diagram, not a control. (c) makes the claim testable in the only way that means anything: hold the correct password, present no second factor, and read nothing. That is asserted for all eight principals across seven consumer views in `tests/test_auth.py` . **Reason for the shape, which is the subtle part.** Identity still comes from `session_user` , which no statement can forge. The session token is a *second* thing, and a bearer secret, so it is bound to `pg_backend_pid()` : a copied token is useless on another connection, and one user completing a login does not authorise every other connection open as the same role. The session check is ANDed over the whole of `sec.v_my_clients` rather than inside one branch, so a branch added later cannot quietly skip it.

CONSEQUENCE Every consumer path now logs in -- the question runner, the workbook, the worked example and the tests. That is more machinery than a demo strictly needs, and it is the price of the control being real rather than described. It also costs wall-clock time: a code may be used once, so two logins as the same principal inside thirty seconds wait for the next code. Weakening that to make a script faster would be removing the control in order to demonstrate it.

D027 · Derive the grant set; store only what a human decided

CONTEXT The first implementation of `sec.access_grant` was a table, rebuilt on each pipeline run from the banking records and the portfolios.

OPTIONS (a) Keep the materialised table and rebuild it. (b) Derive the derivable sources live in a view, and store only delegations.

CHOICE (b), after (a) failed a test.

CONSEQUENCE Ownership, disposition and portfolio rights are computed from the records that already define them, so there is nothing to keep in step. `sec.access_delegation` stores the one source a human chooses, and demands a grantor, a reason and usually an expiry -- because it is the only source that can be wrong in a way the banking records cannot reveal.

After delivery: D028–D029 (not in the delivered version)

After delivery

These two decisions were made on 2026-10-28 on branch `ccr-46f6f45a-t9wtd4` and deliberately not merged into the delivery freeze, because D028 changes a certified number. Present them as what you did next, never as part of what was submitted.

Semantic correctness and balance integrity (D028-D029)

D028 · The closing posting of a day is the end of its running-balance chain

CONTEXT D019 broke ties between postings on the same date with the highest `trans_id` , on the belief that the source records no intra-day order. It records no time and no sequence, but it does record the running balance: each posting starts at `balance_after - signed_amount` and ends at `balance_after` . Measured over all 185,057 posted account-months, the postings of the closing day form a chain with a single end in 154,713 cases, close on a single posting in 28,041, and do not resolve in 2,303. Where the chain resolves, the highest `trans_id` picks the true closing balance 694 times (0.4%), the lowest 96.9%, and file order about half the time. Interest postings carry high ids and are posted before the statement fee, so the old rule read the balance before the fee: account 31 closed November 1998 at 13,312.90 instead of 13,298.30.

OPTIONS (a) Keep the highest-id tie-break. (b) Use file order. (c) Take the end of the balance chain, and use a documented fallback only where the chain does not resolve, marking those rows. (d) Report NULL where ambiguous.

CHOICE (c), with the lowest `trans_id` among the chain ends as the fallback and `closing_basis` ('single posting' / 'balance chain' / 'ambiguous') on every month-end row.

REASON It implements A14 ("the balance after the last posting") with the only evidence of order the source holds, instead of a proxy that is wrong 99.6% of the time. (d) would silently shrink Q9's population; (a) and (b) fabricate an order. The fallback is a measured heuristic, not a claim of chronology, so the 1.2% of account-months it decides stay visibly marked.

COST Q9 moves from 43,808.94 to 43,810.47. The same correction applies to the two reconciliation rules, which read the "last reported balance" the same way: material breaks fall from 55 to 14 and the rounding band from 81 to 70, because 52 of the old flags were a mid-day balance read as the final one. The data owner should confirm the fallback for the 2,303 ambiguous account-months.

CONSEQUENCE `tests/test_month_end.py` runs the production SQL block against synthetic postings (same-day chains, gaps, first and last months, ambiguity) and checks the real table against an independent derivation.

D029 · Generated SQL must match a declared intent before it runs

CONTEXT The guard decides whether a statement may run, never whether it answers the question. A query for "male or female loans in Q1 1995" passed the guard and failed in the database on `t2.gender`, a column `ai.account_owner` does not have (it has `owner_gender`); worse queries, a wrong quarter or a fan-out join, would have run and returned plausible numbers.

OPTIONS (a) More prompt text. (b) EXPLAIN the query before running it. (c) Make the model state its intent as a small intermediate representation (IR) and verify the SQL against the IR, the semantic layer and the database before execution. (d) Compile SQL from the IR and drop model SQL entirely.

CHOICE (c), with (b) as part of it: the IR is checked against the certified metrics, every column the SQL references is resolved against the semantic layer, the period filter is evaluated day by day against the IR's half-open ranges, joins must be declared and must not fan out the counted entity, required grouping and filters must be present, and PostgreSQL plans the query (EXPLAIN) before it executes. After execution, a certified metric is recomputed from the IR alone and must agree before the answer is shown. Anything missing, malformed or unverifiable fails with the reason.

REASON (a) is advice, not a control. (d) is the long-term target for certified metrics but removes the model from questions it handles well today. (c) catches every failure class seen so far while keeping the model's SQL.

COST Questions outside the certified metrics are refused rather than answered unverified, and some correct but unusual SQL is rejected and retried. One metric per answer: a second measure in the same SELECT runs but is not verified. A verified answer proves the SQL computes the declared intent with the certified definitions; whether the intent is what the user meant is shown, not proven, which is why the IR is reported beside the answer.

CONSEQUENCE `src/sql_ir.py` is the only verifier. The question runner and the web app reach it through one function, `run_questions.verify_and_execute`: guard, IR, columns, consistency, then (only then) the two-factor login, EXPLAIN, guarded execution and recomputation on the caller's own RLS-bound connection. A refusal records the stage and the reason, keeps the SQL, returns no rows, and is fed back for the single retry. Only a fully verified answer is recorded as `answered` and evaluated. `tests/test_sql_ir.py` covers the observed failure, quarters, date columns, fan-out, grouping, malformed IRs and both application paths with a scripted model; the brief's ten recorded answers still verify.

19 Assumption register

Every assumption of the delivered version, A1–A46, generated from the delivered `ASSUMPTIONS.md`, with what changes if it is wrong. A36 is superseded by A39. Business definitions awaiting confirmation: A4 (problematic = B or D), A7 (interest counts as incoming), A19 (CZK only), and calendar rather than fiscal quarters.

Question interpretation

ID	Scope	Assumption	Why	What changes if wrong
A1	Q1	"Region" is the region of the **client's own district** (`client.district_id` → `district.A3`), not of the district their account is registered in.	The question says "clients ... per region", so the client is the subject and their own district is their region.	Material: 409 of 4,500 owners (9.1%) live in a different district than their account. Using the account district moves those clients between regions. Q5 deliberately uses the other reading (A8).
A2	Q1	**All** clients are counted, including any who hold no account.	The question says "clients", not "clients with an account".	None in this copy: every one of the 5,369 clients has at least one disposition (rule `RI_CLIENT_WITHOUT_DISP` reports 0). The flagged count would be published if it were not 0.
A3	Q2	"Granted per year" is dated by `loan.date`, the loan start date, and "total amount" is the sum of the **principal**.	It is the only date the loan table carries. Principal is what "amount of loans granted" means; `payments * duration` is the gross repayment, a different number.	Using `payments * duration` raises every yearly total by the interest, roughly 10-20%.
A4	Q3, Q10	"Problematic" = loan status **B** (finished, not repaid) or **D** (running, client in debt).	The published status meanings: A and C are healthy, B and D are not. Materialised once as `core.fact_loan.is_problematic`.	Counting only B (realised losses) gives 31 of 682 = 4.55% instead of 76 of 682 = 11.14%. The definition is the answer, which is why it lives in the model and not in a query.
A5	Q4, Q8	"The card of the account owner" is the card on the **owner's** disposition. A card held by a disponent on the same account is not attributed to the owner.	Both questions say "of the account owner" / "account owners holding". A disponent is not the owner.	0 rows differ in this copy (rule `CON_CARD_HELD_BY_DISPONENT` reports 0), but the rule and the view enforce it so a future feed cannot change the answer silently.
A6	Q4	Loans whose owner holds **no card** are reported as an explicit `no card` category, not dropped.	Dropping them would make Q4's loan counts disagree with Q2 and Q3 over the same 682 loans.	The `no card` group holds the large majority of loans; dropping it would change the reported averages into a statement about card holders only.
A7	Q5	"Incoming" = transaction type `PRIJEM` (`direction = 'credit'`), summed on the **unsigned** amount.	The question asks for incoming volume, not a net flow.	Netting credits against debits would rank districts by net accumulation, a different business question.
A8	Q5	The district is the district the **account** is registered in, not the owner's own district.	The money arrives at an account, not at a person.	Material for the same 9.1% as A1. Stated explicitly because Q1 and Q5 deliberately use different districts.
A9	Q6	All twelve months of 1996 are reported, with zero where no account was opened, by driving the query from `core.dim_date`.	An omitted month reads as "no data", which is a different claim from "no accounts".	Every month of 1996 has accounts in this copy, so the zero-fill does not change a number here -- it changes whether the answer is trustworthy on a different copy.
A10	Q7	Age is **exact**: date of birth to loan start in years including the fractional part.	"Average age at loan start" is a duration, and whole-year differences bias the average by about half a year.	Truncating to completed years (`date_part('year', age(loan, birth))`) reports ages about 0.5 years lower (36.59 vs 37.07 for status A). A calendar-year difference (`year(loan) - year(birth)`) lands within 0.11 years of the exact average per status, but is wrong for any individual loan. (Corrected 2026-10-28: an earlier version said the calendar-year difference reads 0.5 years high; measured, it does not.)
A11	Q7	The owner is the `OWNER` disposition of the loan's account; owners whose birth number does not decode are excluded and the covered population is reported.	A loan has no person of its own; it reaches one through the account owner.	0 owners are excluded in this copy (all 5,369 birth numbers decode). 6 loans carry an owner recorded as under 15, so Q7 reports the average with and without them (rule `VAL_OWNER_AGE_AT_LOAN_START`, warn).
A12	Q8	The population is account **owners** whose own disposition holds a gold card.	As A5.	See A5.
A13	Q8	The unit counted is the **owner row** (one per account), with `count(distinct client)` reported beside it.	A client owning two accounts, each with a gold card, is two card-holder relationships but one person.	Both numbers are published, so neither reading is hidden: in this copy they are equal (53 male / 35 female, 88 distinct clients), so the ambiguity happens not to bite.
A14	Q9	The balance at 1998-12-31 is the balance after the **last posting on or before** that date, from `core.fact_account_month_end_balance`, with ties on a date broken by the **highest** `trans_id`.	`trans_id` is not chronological within a day, so a tie-break is unavoidable; it is documented and applied identically in the reference SQL, in the model and in the semantic layer.	183 accounts have several postings on their latest date (rule `CON_INTRADAY_ORDER_AMBIGUOUS`, warn), so for those the closing balance is a documented choice rather than a fact.
A15	Q9	The average is over accounts with a **known** balance on that date; an account with no posting is excluded, not counted as zero.	A zero would be a claim about the account's balance that the data does not support, and it would bias the average downwards.	0 accounts are excluded in this copy (all 4,500 have postings), so the answer covers the whole book.
A16	Q10	The share is problematic loans divided by **all loans of accounts with that statement frequency** (a within-frequency rate), not each frequency's share of all problematic loans.	"Which frequency has the highest share of problematic loans" is a rate per frequency.	The other reading ranks by volume and would favour the monthly frequency, which carries most accounts. The full ranking with loan counts is returned so a small denominator is visible.

Pipeline and environment

ID	Scope	Assumption	Why	What changes if wrong
A17	Dates	A two-digit source year is always `19xx`.	The dataset runs 1993-1998 and was published in 1999; there is no 20xx data.	A `20xx` reading would move dates out of the validity window, which rule `VAL_DATE_OUT_OF_RANGE` would block -- so the assumption fails loudly rather than silently.

ID	Scope	Assumption	Why	What changes if wrong
A18	Gender	`birth_number` encodes sex as month + 50 for women, and that is the only available source for gender.	Documented with the dataset; `CON_GENDER_MATCHES_BIRTH_NUMBER` checks the decode against the raw digits on every run.	Q8 is entirely built on it. The rule turns a decoder regression into a build failure.
A19	Currency	All amounts are in the same currency (CZK) and no conversion is needed.	Single-country dataset from one bank; no currency column exists.	If a second currency appeared, every sum (Q2, Q4, Q5, Q9) would be adding unlike units. There is no rule that could detect it from this schema, which is why it is written down.
A20	Rounding	Source amounts carry one decimal place, and the per-account balance chain is therefore reconciled with a tolerance (1 CZK rounding band, 100 CZK materiality band) rather than exactly.	Measured: every one of 1,056,320 amounts has a zero in the second decimal.	631 of 4,500 accounts reconcile exactly, 4,364 within 1 CZK, 55 are off by more than 100 CZK. Without the bands the rule would report 3,869 "defects" and hide the 55 that matter.
A21	Entitlements	A user entitled to a client sees that client, the accounts that client **owns**, and everything hanging off those accounts. A disponent relationship does not grant the portfolio holder access to the account.	"Each user sees only rows for their own clients": ownership is the relationship a relationship manager is accountable for.	Including disponent relationships would widen every portfolio; `sec.portfolio_client` plus the policy predicate is where that choice is changed, and it is one SQL predicate, not a model change.
A22	Entitlements	Reference data (`dim_district` , `dim_date` , `ref_codes`) is **not** entitlement filtered.	They contain no client information, and filtering them would break every group-by for every user.	If districts were considered sensitive, the policy would be added to those tables too; nothing else in the design changes.
A23	Data copy	The public Berka copy is the primary dataset, because `data/raw/provided/` is empty. Its row counts match the published reference exactly, so it carries no injected defects.	No bank-supplied copy was received (decision point HD1).	On the bank's copy the counts would differ, `REC_ROWCOUNT_RAW_TO_STG` and `REC_LOAN_STATUS_DISTRIBUTION` would fire, and the same 123 rules would run unchanged against `data/raw/provided/` .
A24	LLM	The LLM is treated as an untrusted query author: it sees the semantic layer and the question, never the database, and its SQL is parsed and rewritten before execution.	It is the only assumption under which "an LLM-generated query cannot bypass the filter" can be true.	If the model were trusted, the entitlement filter would have to live in the prompt -- which is a request, not a control.

Cleaning and extraction

ID	Scope	Assumption	Why	What changes if wrong
A25	C03	The placeholder list (`''` , `'?'` , `'NA'` , `'N/A'` , `'NULL'` , `'-'` , `'unknown'`) is the complete set of ways this source writes "missing".	Derived by profiling every text column, not assumed up front; the profiler reports the full distinct-value distribution per column and no other placeholder-shaped value appeared.	An eighth spelling would be counted as a real value, overstating completeness for that column. It would not corrupt anything downstream, and the profile report is where it would show up. Low risk, cheap to extend: one tuple in `src/config.NULL_PLACEHOLDERS` .
A26	C03, C09	A NULL after cleaning means "the source did not record this", never "the pipeline lost it".	Every value the cleaning dropped or changed is kept in a `<column>_raw` column and listed in `_conversion_notes` , so the two cases are distinguishable by construction.	This is the assumption the whole ledger exists to make checkable. If it were false, every completeness number would be measuring the loader rather than the source.
A27	C09	`operation_code IS NULL` on a transaction means a bank-generated posting, not a missing value.	Measured: all **183,114** rows with no operation carry `k_symbol = UROK` (interest credited), and no other category has a NULL operation. The bank books interest without a customer-facing operation.	If some of those NULLs were genuinely missing data, the completeness rule on that column would have to be `warn` rather than `info` , and "transactions by operation" would be understating one category. The measurement is re-run every build, so a mixed population would show up as UROK no longer being the only value.
A28	C09, D017	The code sets are **open**: the source may contain a code the catalogue does not know, and the right response is to fail the build rather than guess a label.	It already happened. `LEASING` appeared on 341 standing orders and was not in the map; because the catalogue check did not cover that table, nothing failed and the rows shipped unlabelled.	The assumption now has teeth: `check_codes_against_staging` covers every coded column and `tests/test_dq.py::test_every_present_code_resolves_to_a_label` fails if any staged row carries a code with no label. A new code stops a build, which is the intended cost.
A29	C10	`PRIJEM` is the only credit type; `VYDAJ` and `VYBER` are both debits.	Documented with the dataset, and consistent with the reported running balance: the balance chain reconciles for 4,364 of 4,500 accounts under this sign convention and would reconcile for almost none under any other.	Every monetary answer that nets or directs money (Q5, Q9) depends on it. The balance reconciliation rule is the check: a wrong sign convention would make it fail at scale rather than for 55 accounts.
A30	C10, C20	`balance_after` as reported by the bank is authoritative; the pipeline never recomputes it from the amounts.	It is the bank's own record of the account state after the posting. Recomputing would substitute this pipeline's arithmetic for the bank's ledger.	The 55 accounts whose chain does not reconcile beyond 100 CZK are reported as a defect rather than corrected (D013). If the bank's balances were known to be unreliable, the derived balance would become the published one and those 55 would become silent.
A31	C19, C20	The extract is a point-in-time snapshot with no history: no attribute of a client, account or district changes over time within it.	There is no effective-date column anywhere in the eight files, and no table carries two versions of the same key.	No slowly changing dimensions are needed, which is why `dim_client` and `dim_district` are plain. A feed with history would need SCD2 on at least district attributes, since the 1995/1996 socio-economic columns are already two snapshots flattened into one row.
A32	C19	The dataset window is 1993-01-01 to 1998-12-31, and `dim_date` is generated to exactly that span.	Measured from the data: the earliest and latest transaction dates are exactly those two days.	A date outside the window would not join to `dim_date` and would drop out of any date-driven aggregate. `VAL_DATE_OUT_OF_RANGE` is the rule that makes that fail loudly rather than quietly.
A33	C16	Row order within a source file carries no meaning; the only ordering is by the date and key columns.	The files are flat extracts with no sequence column beyond the surrogate ids, and `trans_id` is demonstrably not chronological within a day (`CON_INTRADAY_ORDER_AMBIGUOUS` , 183 accounts).	If file order were the true posting order, the Q9 tie-break would have a better answer available and those 183 accounts would stop being ambiguous. `_source_row` is retained on every row, so that reading could be tested without a reload.

Privacy and identity

ID	Scope	Assumption	Why	What changes if wrong
A34	Identity	A self-service customer authenticates as their own database login role, and that role is never shared, pooled or re-pointed.	Same reasoning as A21/D009 for staff: `session_user` cannot be changed by any statement the caller or an LLM emits, because changing it needs a password neither ever sees.	This is the load-bearing assumption of the whole privacy claim. If customer logins were pooled behind one shared role, the entitlement would resolve to that shared role and every customer would see every other customer's data. Connection pooling is therefore named in the DDR as the first thing that must be redesigned, not quietly added.

ID	Scope	Assumption	Why	What changes if wrong
A35	Identity	A customer is entitled to exactly the client record that `sec.app_user.subject_client_id` names, and the mapping from a real person to that client id is correct and is made outside this system.	This dataset has no identity documents, no logins and no names; the mapping from "the person at the counter" to `client_id` is the bank's KYC process, not a data-product concern.	The product enforces the mapping it is given. A wrong mapping shows the wrong person's data with the full force of a correct-looking entitlement, and nothing in this build could detect it. In a real deployment the insert into `sec.app_user` is the output of an identity-proofing process and should be auditable on that basis.
A36(superseded by A39 / D023)	Identity	A customer sees the accounts they **own**, not the accounts they may merely operate as a disponent.	Consistency with A21 and the safer default for a privacy boundary (D022).	869 clients hold a disponent relationship. Under the other reading each of them would additionally see that account and its postings. It is one predicate in `sec.v_my_accounts` and it is a business decision, not a technical one.
A37	Privacy	Reference data (districts, calendar, code catalogue) is not sensitive, so a customer may read all 77 districts even though they belong to one.	Same as A22: filtering it would break every group-by, and a district list reveals nothing about any client.	If district-level data were considered identifying in combination with other attributes, the policy would extend to it. Nothing else in the design changes.
A38	Privacy	No read audit is required for this submission.	The brief asks who *can* see what, not what was actually read.	For a real bank this is a regulatory gap, not a simplification. It is listed in the worked example's limits and in the build review as the largest missing control.

Identity, access and authentication

ID	Scope	Assumption	Why	What changes if wrong
A39	Access	A customer may see every account they are authorised to OPERATE, including accounts they do not own.	What the bank tells its customers their app will show. Supersedes A36, which narrowed them to ownership; the decision was taken to the business reading and settled (D023).	869 clients hold a disposition on an account they do not own. Under the narrow reading each of them sees one fewer account. It is one branch in `sec.v_access_grant`.
A40	Access	A disposition grants the ACCOUNT and nothing else -- not the owner's client record, not their other accounts.	Authorising someone to pay a bill from your account is not authorising them to read you. Nobody signing that arrangement would expect otherwise.	If a disposition granted the owning client instead, every disponent would see their co-holder's full customer record. This is the single most consequential line in the access model, and it is tested by name.
A41	Access	A staff portfolio grants `view` and never `operate`.	Being accountable for a client is not authority to move their money.	Nothing acts on `operate` in this read-only product, so the distinction costs nothing today and cannot be recovered later if it is dropped now. A test pins it.
A42	Identity	An opaque `client_ref` is sufficient protection for an identifier that appears in a URL or on a statement; it does not need to be rotated per session.	It is unguessable without the salt, and row level security stops access regardless. The reference prevents enumeration and correlation, not access.	If a reference leaked in bulk, rebuilding under a new salt reissues all of them. Per-session identifiers would be stronger and would make every support conversation impossible.
A43	Authentication	A time-based one-time code is an adequate second factor for this demonstration.	It is the standard most people already have an app for, and it is verifiable offline against RFC 6238.	TOTP is phishable in real time: a victim relaying a live code to an attacker defeats it. A deployment handling real money should prefer a phishing-resistant factor (WebAuthn/passkeys). The session model here would not change; only the factor would.
A44	Authentication	Storing TOTP shared secrets in a recoverable form is acceptable here.	Verification requires the secret, so it cannot be hashed.	`sec.auth_factor.secret` is the most sensitive column in the schema. A real deployment encrypts it with a key held outside the database, so that a dump of the data is not a dump of everyone's second factor.
A45	Authentication	Binding a session to `(db_role, pg_backend_pid)` is sufficient to stop replay.	A stolen session id is useless on a different connection, and the holder cannot forge `session_user` either.	Backend pids are reused after wraparound. A replay would need the same pid, the same role and the session still inside its 30-minute window; the combination is thin but not impossible, and a production system would add the backend start time to the binding.
A46	Demo	Printing the one-time code in the demo console does not weaken the claims being demonstrated.	The control being shown is what the second factor *stops*, not where the code comes from; every other part of the path is the production one.	A system that can display your own second factor has one factor. It is marked as a demo affordance at every call site and is the first thing a deployment removes.

Added after delivery (not in the delivered version)

ID	Scope	Assumption	Why	What changes if wrong
A47	E1 (gender of loans)	"Male or female loans" counts **loans**, once each, grouped by the gender of the loan's **account owner** (`ai.loan.owner_gender` , i.e. `core.dim_client.gender` through the `OWNER` disposition). **Pending business confirmation.**	Gender is an attribute of a client only (`ai.client.gender`); `ai.loan` and `ai.account_owner` carry it as `owner_gender` . A loan has no borrower of its own in the source; it reaches a person through its account, and A5/A11 already attribute loans to the owner. Every account has exactly one owner, so the join cannot repeat a loan.	Q1 1995: 28 loans, 18 male-owned and 10 female-owned. Counting every client on the account (owners and disponents) counts loan-client pairs, not loans: 34 pairs, because 6 loans have a disponent of the other gender, so the two groups would overlap. The verifier refuses that join path for `loan_count` unless the metric is redefined.
A48	E1-E4 (quarters)	"Q1"-"Q4" in a question are **calendar** quarters (Jan-Mar, Apr-Jun, Jul-Sep, Oct-Dec), written as half-open date ranges, and a comparison across periods returns **one row per period**. Q1-Q10 in the context pack only label the reference questions. **Pending business confirmation.**	The book has no fiscal calendar and nothing in the source defines one. One row per period is what the verifier can check against the IR and what the evaluator compares.	A fiscal year starting in another month moves every quarter boundary; it would be one change to `ai.calendar` and the IR periods, and the day-by-day period check would then enforce the new boundaries. A pivoted answer (one column per period) is refused today and would need the verifier extended.
A49	Q5 (interest)	Bank-credited **interest** (`category_code = 'UROK'` , 48,788 credits in 1997) counts as incoming money. **Pending business confirmation.**	A7 defines incoming as every `PRIJEM` credit, and interest is one. The previous audit listed this as an open decision; nothing in the source settles it.	Measured for 1997: interest is 7,372,897.70 of 864,692,326.70 CZK incoming (0.85%). Excluding it lowers every district's total by about that share and leaves the top-10 order unchanged.

20 Glossary and where to find things

Terms

Term	Meaning in this project
Medallion (bronze / silver / gold)	Layering by trust: verbatim (raw), typed and graded (stg + dq), modelled and published (core + ai)
Grain	What one row means ("one row per loan"). A false grain silently inflates every count
Fan-out	Joining to the "many" side of a relationship, which repeats rows and inflates sums and counts
Disposition / owner / disponent	The link between a client and an account. `OWNER` holds the account (exactly one per account); a `DISPONENT` may operate it but does not own it
Quarantine	Where a `block` row goes: excluded from core and the ai views, with a reason, reversible
Flag	A `warn` marker on a kept row (`dq.flags` , `_dq_flags`) that travels to the answer as a caveat
Reconciliation	Proof that a layer lost or invented nothing: counts and money sums compared between layers
Semantic layer	The YAML file of entities, grains, metrics, joins, glossary and caveats; validated against the database and rendered into the **context pack** the model reads
Certified metric	A metric defined once in the semantic layer (e.g. `problematic_loan_share`) that the model must use verbatim
Row level security (RLS)	A PostgreSQL policy that filters the rows a role may see, inside every statement's plan
`security_invoker`	A view option that evaluates the underlying policies as the *caller*, not as the view's owner
`session_user`	The role that authenticated the connection; cannot be changed by SQL without a password
TOTP	Time-based one-time password (RFC 6238), the second factor, verified inside the database
Guard	`src/sql_guard.py` : parses model SQL and enforces its *shape* (one read-only SELECT on `ai` , bounded)
IR / intent verification	(After delivery) the model states metric, periods, grouping and filters; the SQL is checked against that statement before and after execution
Tie-break	The rule that picks the closing posting when several share a date; delivered: highest `trans_id`
Balance chain	Each posting's opening balance is `balance_after - signed_amount` ; postings of a day link into a chain whose end is the closing posting
Value-based evaluation	Comparing the numbers an answer contains, not its SQL text or column names

Codes worth knowing

Source code	Meaning
`PRIJEM`	Credit (incoming)
`VYDAJ` , `VYBER`	Debit (withdrawal); `VYBER` is not in the data description but is a debit
`UROK`	Interest credited (bank-generated; operation NULL)
`SANKC. UROK`	Sanction interest if negative balance
`SLUZBY`	Payment for statement (the fee)
`POPLATEK MESICNE` / `TYDNE` / `PO OBRATU`	Statement issued monthly / weekly / after each transaction
Loan status A / B / C / D	Finished paid / finished unpaid / running OK / running in debt — B and D are "problematic"
Birth number `YYMMDD`	Women carry month + 50

Where to find things in the repository

You need	Look in
Every claim and its evidence	`docs/DELIVERY_AUDIT.md`
The rules, severities and reasons	`dq/dq_rules.yaml` ; last run: `dq/results/dq_report.md`
What the cleaning did, measured	`docs/CLEANING_LEDGER.md`
The model narrative and row counts	`deliverables/model_explanation.md`
The semantic layer / the text the model reads	`semantic/semantic_layer.yaml` / `semantic/context_pack.md`
The guard	`src/sql_guard.py`
The policies / authentication	`sql/50_sec_rls.sql` / `sql/51_sec_auth.sql`
Reference SQL and results	`sql/60_reference_queries.sql` / `dq/results/reference_results.json`
Claude's answers and the evaluation	`dq/results/llm_results.json` , `evaluation.json` , `llm_logs/`
The local model's run	`dq/results/local/` , `llm_logs/local/`
Decisions / assumptions	`DECISIONS.md` / `ASSUMPTIONS.md`
The AI disclosure	`AI_DISCLOSURE.md`

You need	Look in
Security design	`docs/ACCESS_AND_AUTH.md` ; one client end to end: `docs/WORKED_EXAMPLE.md`
Running it on Windows with Ollama	`docs/LOCAL_LLM_WINDOWS.md`

This study pack supersedes the older walkthrough prep

`docs/INTERVIEW_PREPARATION.md` was written before the delivery freeze and still quotes some older figures (PostgreSQL 16 only, 241 tests). Where it and this pack differ, this pack has been checked against the delivered commit.